



ESPE
UNIVERSIDAD DE LAS FUERZAS ARMADAS
INNOVACIÓN PARA LA EXCELENCIA

**UNIVERSIDAD DE LAS FUERZAS ARMADAS
ESPE**

**DEPARTAMENTO DE CIENCIAS DE LA
COMPUTACIÓN**

CARRERA: INGENIERÍA DE SOFTWARE

NRC: 30724

ASIGNATURA: Análisis y Diseño de Software

TEMA: Informe Plan de Pruebas - Sistema Fi-Citas

Nombres (Equipo de Pruebas):

- Altamirano, Xavier
- Cordero, Martin
- Ayo, Alisson

DOCENTE: Ing. Jenny Ruiz

FECHA: 8 de Julio del 2026

Sistema Fi-Citas - Agendamiento Centro Mónica Viteri

Proyecto: Fi-Citas - Sistema Web de Gestión de Citas
Tipo de Documento: Plan de Pruebas Unitarias, Integración y Sistema
Fecha de Creación: Junio 2026
Fecha de Ejecución: 8 de Julio del 2026
Versión: 1.0 (Metodología Scrum)
Estado: COMPLETADO (100 %)

RESUMEN EJECUTIVO

Estado de Implementación

Métrica	Valor
Tests Ejecutados	109/109 (100 % pasando)
Tiempo de Ejecución	4.215s
Suites de Tests	7/7 (100 % pasando)
Requisitos Implementados	7/7 (100 %)
Tasa de Éxito	100 %

Requisitos Funcionales - Estado Final

RF	Nombre	Estado	Tests	Tasa Éxito
REQ001	Inicio de Sesión	COMPLETO	12	100 %
REQ002	Recuperación de Contraseña	COMPLETO	4	100 %
REQ003	Gestión de Pacientes	COMPLETO	32	100 %
REQ004	Gestión de Perfiles	COMPLETO	9	100 %
REQ005	Gestión de Citas	COMPLETO	38	100 %
REQ006	Robustez y Manejo de Errores	COMPLETO	10	100 %
REQ007	Generación de Notificaciones	COMPLETO	4	100 %

Índice

1. OBJETIVO DE LAS PRUEBAS	1
1.1. Objetivo General	1
1.2. Objetivos Específicos	1
2. PLANTEAMIENTO	1
2.1. Alcance de las Pruebas	1
2.2. Enfoque de Testing	2
2.3. Estrategia de Mocking	2
3. HERRAMIENTA	2
3.1. Framework de Testing	2
3.2. Entorno de Pruebas	2
3.3. Estructura de Archivos	3
4. REQ001 - Inicio de Sesión	4
4.1. Descripción	4
4.2. Archivos involucrados	4
4.3. Casos de prueba ejecutados	4
4.3.1. Backend	4
4.3.2. Frontend	5
4.4. Resumen de ejecución	5
4.5. Implementación	5
4.5.1. Prueba 1. Validación de solicitud de inicio de sesión sin credenciales	5
4.5.2. Prueba 2. Validación de credenciales incorrectas	5
4.5.3. Prueba 3. Validación del flujo completo para impedir citas solapadas	6
4.5.4. Prueba 4. Validación del cambio de contraseña sin datos de entrada	7
4.5.5. Prueba 5. Validación de longitud mínima para la nueva contraseña .	8
4.5.6. Prueba 6. Cambio exitoso de contraseña	8
4.5.7. Prueba 7. Validación de contraseña actual incorrecta	9
4.5.8. Prueba 8. Desbloqueo automático de usuario	9
4.5.9. Prueba 9. Autenticación de un administrador válido	10
4.5.10. Prueba 10. Renderizado del formulario de inicio de sesión	11
4.5.11. Prueba 11. Envío del formulario de autenticación	11
4.5.12. Prueba 12. Visualización de mensajes de error durante la autenticación	12
4.6. Resultados	13
5. REQ002 - Recuperación de Contraseña	14
5.1. Descripción	14
5.2. Archivos involucrados	14
5.3. Casos de prueba ejecutados	14
5.3.1. Frontend	14
5.4. Resumen de ejecución	14
5.5. Implementación	14
5.5.1. Prueba 1. Visualización del panel de cambio de contraseña	15
5.5.2. Prueba 2. Validación de coincidencia entre contraseñas nuevas . . .	15
5.5.3. Prueba 3. Actualización exitosa de contraseña	16
5.5.4. Prueba 4. Manejo de errores durante el cambio de contraseña	19

5.6. Resultados	21
6. REQ003 - Gestión de Pacientes	21
6.1. Descripción	21
6.2. Archivos involucrados	21
6.3. Casos de prueba ejecutados	21
6.3.1. Pruebas Unitarias	21
6.4. Implementación	22
6.4.1. Prueba 1. Registro de un paciente con información válida	22
6.4.2. Prueba 2. Validación del campo obligatorio nombre	22
6.4.3. Prueba 3. Validación de edad negativa	23
6.4.4. Prueba 4. Validación de edad máxima permitida	23
6.4.5. Prueba 5. Validación del peso corporal	24
6.4.6. Prueba 6. Validación de la estatura	24
6.4.7. Prueba 7. Registro con campos opcionales vacíos	25
6.4.8. Prueba 8. Búsqueda de un paciente inexistente	25
6.4.9. Prueba 9. Consulta del listado de pacientes	26
6.4.10. Prueba 10. Validación de datos obligatorios durante el registro de pacientes	26
6.4.11. Prueba 11. Flujo completo de registro de paciente y agendamiento de cita	27
6.4.12. Prueba 12. Agendamiento exitoso de una cita en un horario disponible	28
6.4.13. Prueba 13. Actualización de antecedentes médicos de un paciente .	29
6.4.14. Prueba 14. Obtención de la lista de pacientes	30
6.4.15. Prueba 15. Consulta de un paciente mediante su identificador . . .	31
6.4.16. Prueba 16. Actualización de la información de un paciente	31
6.4.17. Prueba 17. Eliminación de un paciente	32
6.4.18. Prueba 18. Manejo de errores internos durante la consulta de pacientes	32
6.4.19. Prueba 19. Validación de los servicios de comunicación para la ges- tión de pacientes	33
6.4.20. Prueba 20. Verificación del renderizado del formulario de pacientes .	35
6.4.21. Prueba 21. Validación de campos obligatorios	36
6.4.22. Prueba 22. Cálculo automático del índice de masa corporal	36
6.4.23. Prueba 23. Registro exitoso de un paciente	37
6.4.24. Prueba 24. Manejo de errores durante el registro	39
6.4.25. Prueba 25. Visualización del listado de pacientes	39
6.4.26. Prueba 26. Búsqueda de pacientes	40
6.4.27. Prueba 27. Apertura del formulario de registro	41
6.4.28. Prueba 28. Eliminación de un paciente	42
6.4.29. Prueba 29. Visualización del detalle de un paciente	43
6.4.30. Prueba 30. Manejo de errores durante la eliminación	44
6.4.31. Prueba 31. Cancelación de la eliminación	45
6.4.32. Prueba 32. Cierre del formulario de registro	46
7. REQ004 - Gestión de Perfiles	48
7.1. Descripción	48
7.2. Implementación	48
7.2.1. Prueba 1. Creación de una cita con paciente y doctor válidos	48
7.2.2. Prueba 2. Validación de solapamiento de citas	49

7.2.3.	Prueba 3. Consulta de doctores registrados	49
7.2.4.	Prueba 4. Prevención de citas solapadas	50
7.2.5.	Prueba 5. Consulta individual de un doctor	51
7.2.6.	Prueba 6. Consulta de un doctor inexistente	52
7.2.7.	Prueba 7. Validación de existencia de paciente y doctor	53
7.2.8.	Prueba 8. Consumo del servicio de doctores	53
7.2.9.	Prueba 9. Confirmación de citas desde el panel administrativo	54
8.	REQ005 - Gestión de Citas	56
8.1.	Descripción	56
8.2.	Implementación	56
8.2.1.	Prueba 1. Validación del campo obligatorio apellido	56
8.2.2.	Prueba 2. Actualización de información del paciente	57
8.2.3.	Prueba 3. Validación del horario de atención	57
8.2.4.	Prueba 4. Validación de fechas anteriores	58
8.2.5.	Prueba 5. Reprogramación de una cita médica	59
8.2.6.	Prueba 6. Validación de los datos enviados al crear una cita	59
8.2.7.	Prueba 7. Validación de solicitudes sin autenticación	60
8.2.8.	Prueba 8. Liberación del horario tras cancelar una cita	61
8.2.9.	Prueba 9. Consulta del listado de citas	61
8.2.10.	Prueba 10. Consulta de una cita mediante su identificador	62
8.2.11.	Prueba 11. Actualización de una cita existente	63
8.2.12.	Prueba 12. Eliminación de una cita	63
8.2.13.	Prueba 13. Manejo de errores durante la consulta de citas	64
8.2.14.	Prueba 14. Disponibilidad de horarios tras cancelar una cita	64
8.2.15.	Prueba 15. Validación del formato de fecha	65
8.2.16.	Prueba 16. Registro correcto de una cita médica	66
8.2.17.	Prueba 17. Validación del horario de atención	67
8.2.18.	Prueba 18. Prevención de solapamiento entre citas	67
8.2.19.	Prueba 19. Rechazo de credenciales inválidas	68
8.2.20.	Prueba 20. Bloqueo de cuenta por múltiples intentos fallidos	69
8.2.21.	Prueba 21. Consumo del servicio de citas	70
8.2.22.	Prueba 22. Visualización del formulario de registro de citas	72
8.2.23.	Prueba 23. Validación de campos obligatorios	72
8.2.24.	Prueba 24. Validación del horario ingresado	74
8.2.25.	Prueba 25. Registro exitoso de una cita desde la interfaz	76
8.2.26.	Prueba 26. Manejo de errores durante el registro de citas	78
8.2.27.	Prueba 27. Actualización del estado de una cita desde la interfaz	80
8.2.28.	Prueba 28. Manejo de errores durante la actualización del estado	82
8.2.29.	Prueba 29. Cancelación de una cita	84
8.2.30.	Prueba 30. Manejo de errores durante la cancelación de una cita	86
8.2.31.	Prueba 31. Cancelación abortada por el usuario	88
8.2.32.	Prueba 32. Reprogramación de una cita	89
8.2.33.	Prueba 33. Manejo de errores durante la reprogramación de una cita	91
8.2.34.	Prueba 34. Filtrado de citas por estado y búsqueda	93
8.2.35.	Prueba 35. Apertura del formulario para registrar una nueva cita	95
8.2.36.	Prueba 36. Bloqueo temporal de cuenta tras múltiples intentos fallidos	96
8.2.37.	Prueba 37. Renderizado del componente Modal	98

8.2.38. Prueba 38. Cierre del componente Modal	100
9. REQ006 - Robustez y Manejo de Errores	102
9.1. Descripción	102
9.2. Implementación	102
9.2.1. Prueba 1. Verificación del estado del servicio	102
9.2.2. Prueba 2. Manejo de rutas inexistentes	103
9.2.3. Prueba 3. Consulta de un paciente inexistente	103
9.2.4. Prueba 4. Consulta de una cita inexistente	104
9.2.5. Prueba 5. Validación de identificadores inválidos en el controlador de citas	104
9.2.6. Prueba 6. Validación de identificadores inválidos en el controlador de pacientes	105
9.2.7. Prueba 7. Visualización de indicadores del panel principal	106
9.2.8. Prueba 8. Manejo de errores al confirmar una cita	108
9.2.9. Prueba 9. Visualización de estados vacíos en el panel	109
9.2.10. Prueba 10. Visualización del indicador de carga	110
10.REQ007 - Generación de Notificaciones	110
10.1. Descripción	110
10.2. Implementación	111
10.2.1. Prueba 1. Notificación al confirmar una cita	111
10.2.2. Prueba 2. Notificación al reprogramar una cita	112
10.2.3. Prueba 3. Notificación al cancelar una cita	112
10.2.4. Prueba 4. Notificación al completar una cita	113

1. OBJETIVO DE LAS PRUEBAS

1.1. Objetivo General

Validar que todos los requisitos funcionales del sistema web Fi-Citas cumplan con las especificaciones establecidas para el Centro Mónica Viteri, garantizando la correcta funcionalidad de:

- Gestión de acceso de usuarios (inicio de sesión y recuperación de credenciales).
- Gestión de pacientes (creación, consulta, actualización y eliminación de datos básicos).
- Administración de perfiles médicos.
- Sistema de agendamiento de citas con prevención automática de solapamientos.
- Generación de notificaciones de cambio de estado.

1.2. Objetivos Específicos

1. **Validar modelos de datos:** Verificar que las entidades `User`, `Patient`, `Doctor` y `Appointment` cumplan con sus reglas de negocio.
2. **Validar controladores:** Asegurar que los controladores manejen correctamente la lógica de negocio y la gestión de errores HTTP.
3. **Validar vistas:** Confirmar que los componentes de React rendericen correctamente los datos e interactúen adecuadamente con el usuario.
4. **Validar servicios:** Verificar el funcionamiento de la comunicación asíncrona (API REST) entre el frontend y el backend.
5. **Validar reglas de integración:** Asegurar la correcta validación de disponibilidad de horarios para evitar colisiones en la programación.

2. PLANTEAMIENTO

2.1. Alcance de las Pruebas

Las pruebas automatizadas cubren:

- **Modelos:** Validación de esquemas, campos obligatorios y métodos de negocio.
- **Controladores:** Operaciones CRUD, validaciones de seguridad y manejo de excepciones.
- **Vistas:** Renderizado del DOM virtual y captura de eventos de la interfaz.
- **Servicios:** Peticiones HTTP (`get`, `post`, `put`, `delete`) y consumo de endpoints.

2.2. Enfoque de Testing

Metodología: Integración continua bajo el marco de trabajo ágil Scrum.

Tipo de Pruebas: Unitarias, Integración, Componentes y Sistema.

Cobertura Objetivo: 80 % mínimo (Logrado: 100 % de tests pasando exitosamente).

2.3. Estrategia de Mocking

Para aislar los componentes y evaluar su comportamiento sin depender de la base de datos o la red, se utilizó la siguiente estrategia con Jest:

■ Se utilizó:

- `jest.fn()` para simular eventos de la interfaz y funciones callbacks.
- `mockResolvedValueOnce` y `mockRejectedValueOnce` para simular respuestas exitosas o caídas de los servicios asíncronos.
- `jest.spyOn()` para interceptar y evaluar errores internos en los modelos de base de datos.
- Mocks de objetos globales del navegador (`window.confirm`, `window.alert`, `global.fetch`).

3. HERRAMIENTA

3.1. Framework de Testing

Jest / Vitest - Framework de testing integral para JavaScript y React.

Configuración Especial:

```
1 {  
2   "testEnvironment": "jsdom",  
3   "setupFilesAfterEnv": ["<rootDir>/src/setupTests.js"],  
4   "transform": {  
5     "^.+\\.jsx?$": "babel-jest"  
6   }  
7 }
```

3.2. Entorno de Pruebas

- **Node.js:** v18+
- **Entorno DOM:** jsdom (simula navegador para las pruebas de interfaz).
- **Módulos:** ES6 Modules / CommonJS.

3.3. Estructura de Archivos

```
1 tests/
2     backend/
3         01-unitarias.test.js (8 tests)
4         02-integracion.test.js (4 tests)
5         03-sistema.test.js (2 tests)
6         04-aceptacion.test.js (2 tests)
7         05-cobertura.test.js (5 tests)
8     frontend/
9         views/
10             LoginPage.test.jsx (7 tests)
11             PatientForm.test.jsx (5 tests)
12             Patients.test.jsx (8 tests)
13             AppointmentForm.test.jsx (5 tests)
14             Appointments.test.jsx (8 tests)
15         services/
16             auth.test.js (1 test)
17             api.service.test.js (2 tests)
```

4. REQ001 - Inicio de Sesión

4.1. Descripción

El requerimiento **REQ001** corresponde al módulo de autenticación del sistema, responsable de controlar el acceso de los usuarios a la aplicación mediante la validación de credenciales. Las pruebas realizadas verifican el correcto funcionamiento del proceso de inicio de sesión, el manejo de errores de autenticación, el cambio de contraseña, el desbloqueo automático de cuentas y el comportamiento de la interfaz gráfica durante la interacción del usuario con el sistema.

Para validar este requerimiento se ejecutaron pruebas de integración, sistema, cobertura, pruebas unitarias y pruebas de componente, garantizando el correcto funcionamiento tanto del backend como del frontend.

4.2. Archivos involucrados

Archivo	Cantidad de pruebas
02-integracion.test.js	2
03-sistema.test.js	1
05-cobertura.test.js	5
auth.test.js	1
LoginPage.test.jsx	3
Total	12

Cuadro 1: Distribución de casos de prueba del requerimiento REQ001.

4.3. Casos de prueba ejecutados

4.3.1. Backend

N°	Caso de prueba	Estado
1	POST /api/auth/login - Debe fallar sin credenciales	Aprobado
2	POST /api/auth/login - Debe fallar con credenciales incorrectas	Aprobado
3	Flujo completo: intento de cita solapada se bloquea antes de llegar a persistencia	Aprobado
4	POST /api/auth/change-password - Debe fallar sin inputs	Aprobado
5	POST /api/auth/change-password - Debe fallar con contraseña corta	Aprobado
6	POST /api/auth/change-password - Debe cambiar la contraseña exitosamente	Aprobado
7	POST /api/auth/change-password - Debe fallar con contraseña incorrecta	Aprobado
8	POST /api/auth/login - Debe desbloquear usuario si expiró el tiempo	Aprobado
9	should authenticate a valid admin	Aprobado

4.3.2. Frontend

N°	Caso de prueba	Estado
10	should render login form	Aprobado
11	should call login on submit	Aprobado
12	should show error if login fails	Aprobado

4.4. Resumen de ejecución

Total de pruebas ejecutadas	12
Pruebas aprobadas	12
Pruebas no aprobadas	0
Porcentaje de éxito	100 %

Cuadro 4: Resultado de la ejecución del requerimiento REQ001.

4.5. Implementación

En esta sección se presentan los fragmentos de código correspondientes a las pruebas automatizadas implementadas para validar el correcto funcionamiento del módulo de autenticación.

4.5.1. Prueba 1. Validación de solicitud de inicio de sesión sin credenciales

Esta prueba de integración verifica que el servicio de autenticación rechace solicitudes que no contienen las credenciales requeridas. Para ello se envía una petición HTTP POST al endpoint `/api/auth/login` con un cuerpo vacío y posteriormente se comprueba que el servidor responda con el código HTTP **400 (Bad Request)**, indicando además que la operación no fue exitosa.

```
1  it('16. POST /api/auth/login - Debe fallar sin credenciales',  
2    async () => {  
3      const res = await request(app)  
4        .post('/api/auth/login')  
5        .send({});  
6  
7      expect(res.status).toBe(400);  
8      expect(res.body.success).toBe(false);  
9  
10   });
```

4.5.2. Prueba 2. Validación de credenciales incorrectas

El objetivo de esta prueba consiste en comprobar que el sistema impida el acceso cuando el usuario proporciona una contraseña inválida. Se realiza una petición al servicio

de autenticación utilizando un usuario existente y una contraseña incorrecta, verificando que el servidor responda con el código HTTP **401 (Unauthorized)**.

```
1  it('17. POST /api/auth/login - Debe fallar con credenciales
   incorrectas', async () => {
2
3      const res = await request(app)
4        .post('/api/auth/login')
5        .send({
6          username: 'admin',
7          password: 'badpassword'
8        });
9
10     expect(res.status).toBe(401);
11
12 });
```

4.5.3. Prueba 3. Validación del flujo completo para impedir citas solapadas

Esta prueba de sistema valida el comportamiento integral de la aplicación cuando un usuario intenta registrar una cita médica que entra en conflicto con otra previamente registrada. El procedimiento crea inicialmente un médico y un paciente, registra una primera cita y posteriormente intenta registrar una segunda en un horario no permitido. Finalmente se verifica que el sistema rechace la operación devolviendo el código HTTP **400** junto con un mensaje indicando que debe seleccionarse otro horario.

```
1  it('25. Flujo Completo: Intento de cita solapada se bloquea
    antes de llegar a persistencia', async () => {
2
3      const doctor = await Doctor.create({
4          name: 'Dr. House',
5          specialty: 'General'
6      });
7
8      const patient = await Patient.create({
9          nombre: 'X',
10         apellido: 'Y',
11         edad: 30,
12         peso: 70,
13         estatura: 170,
14         sexo: 'Hombre',
15         ocupacion: 'QA'
16     });
17
18     await request(app)
19         .post('/api/appointments')
20         .send({
21             patientId: patient._id,
22             doctorId: doctor._id,
23             fecha: '2030-05-15',
24             hora: '10:00'
25         });
26
27     const res = await request(app)
28         .post('/api/appointments')
29         .send({
30             patientId: patient._id,
31             doctorId: doctor._id,
32             fecha: '2030-05-15',
33             hora: '11:00'
34         });
35
36     expect(res.status).toBe(400);
37     expect(res.body.message)
38         .toMatch(/elige otro horario/);
39
40 });
```

4.5.4. Prueba 4. Validación del cambio de contraseña sin datos de entrada

Esta prueba comprueba que el servicio encargado del cambio de contraseña valide la existencia de los parámetros obligatorios antes de procesar la solicitud. Cuando el cuerpo de la petición se encuentra vacío, el sistema debe responder con el código HTTP 400, indicando que la solicitud no cumple con los requisitos establecidos.

```
1  it('POST /api/auth/change-password - Debe fallar sin inputs',  
2    async () => {  
3      const res = await request(app)  
4        .post('/api/auth/change-password')  
5        .send({});  
6  
7      expect(res.status).toBe(400);  
8  
9    });
```

4.5.5. Prueba 5. Validación de longitud mínima para la nueva contraseña

Esta prueba verifica que el servicio de cambio de contraseña aplique correctamente las reglas de validación sobre la longitud mínima de la nueva contraseña. Se envía una solicitud con una contraseña que no cumple el tamaño requerido y se comprueba que el servidor rechace la operación mediante el código HTTP **400 (Bad Request)**.

```
1  it('POST /api/auth/change-password - Debe fallar con pass corto',  
2    , async () => {  
3      const res = await request(app)  
4        .post('/api/auth/change-password')  
5        .send({  
6          username: 'admin',  
7          currentPassword: '123',  
8          newPassword: '12'  
9        });  
10  
11      expect(res.status).toBe(400);  
12  
13    });
```

4.5.6. Prueba 6. Cambio exitoso de contraseña

Esta prueba valida el funcionamiento correcto del proceso de actualización de contraseña. Inicialmente se registra un usuario de prueba dentro de la base de datos y posteriormente se envía una solicitud con las credenciales actuales y una nueva contraseña válida. El caso se considera exitoso cuando el servidor responde con el código HTTP **200 (OK)**, indicando que la contraseña fue actualizada correctamente.

```
1  it('POST /api/auth/change-password - Debe cambiar la contraseña
    exitosamente', async () => {
2
3      await User.create({
4          username: 'testuser',
5          password: 'oldpassword',
6          role: 'ADMINISTRADOR',
7          displayName: 'User'
8      });
9
10     const res = await request(app)
11       .post('/api/auth/change-password')
12       .send({
13         username: 'testuser',
14         currentPassword: 'oldpassword',
15         newPassword: 'newpassword123'
16       });
17
18     expect(res.status).toBe(200);
19
20 });
```

4.5.7. Prueba 7. Validación de contraseña actual incorrecta

Esta prueba comprueba que el sistema impida el cambio de contraseña cuando la contraseña actual proporcionada por el usuario no coincide con la almacenada en la base de datos. El comportamiento esperado consiste en rechazar la solicitud devolviendo el código HTTP 401 (Unauthorized).

```
1  it('POST /api/auth/change-password - Debe fallar con pass
    incorrecto', async () => {
2
3      const res = await request(app)
4        .post('/api/auth/change-password')
5        .send({
6          username: 'testuser',
7          currentPassword: 'wrong',
8          newPassword: 'newpassword123'
9        });
10
11     expect(res.status).toBe(401);
12
13 });
```

4.5.8. Prueba 8. Desbloqueo automático de usuario

Esta prueba verifica que el mecanismo de bloqueo temporal permita nuevamente el acceso cuando el tiempo de restricción ha expirado. Para ello se registra un usuario cuyo

atributo `lockedUntil` contiene una fecha anterior a la hora actual. Posteriormente se intenta iniciar sesión comprobando que el servidor responda satisfactoriamente mediante el código HTTP **200 (OK)**.

```
1  it('POST /api/auth/login - Debe desbloquear usuario si expiró el
   tiempo', async () => {
2
3      await User.create({
4          username: 'lockeduser',
5          password: '123',
6          role: 'ADMINISTRADOR',
7          displayName: 'Locked',
8          lockedUntil: new Date(Date.now() - 1000)
9      });
10
11     const res = await request(app)
12       .post('/api/auth/login')
13       .send({
14         username: 'lockeduser',
15         password: '123'
16       });
17
18     expect(res.status).toBe(200);
19
20 });
```

4.5.9. Prueba 9. Autenticación de un administrador válido

Esta prueba unitaria valida el proceso principal de autenticación del sistema. Se realiza una petición al servicio de inicio de sesión utilizando las credenciales válidas del administrador y posteriormente se verifica que el servidor responda exitosamente, retornando además la información correspondiente al rol del usuario autenticado.

```
1  it('should authenticate a valid admin', async () => {
2
3      const res = await request(app)
4        .post('/api/auth/login')
5        .send({
6          username: 'admin',
7          password: 'admin123'
8        });
9
10     expect(res.statusCode).toBe(200);
11     expect(res.body.success).toBe(true);
12     expect(res.body.data.role).toBe('ADMINISTRADOR');
13
14 });
```


4.5.10. Prueba 10. Renderizado del formulario de inicio de sesión

Esta prueba de componente verifica que la interfaz gráfica muestre correctamente todos los elementos necesarios para que el usuario pueda autenticarse. Se valida la presencia del título de la aplicación, así como los campos destinados al ingreso del usuario y la contraseña.

```
1  it('should render login form', () => {  
2  
3      render(<LoginPage />);  
4  
5      expect(  
6          screen.getByText('FiCitas')  
7      ).toBeInTheDocument();  
8  
9      expect(  
10         screen.getByLabelText('Usuario')  
11     ).toBeInTheDocument();  
12  
13     expect(  
14         screen.getByLabelText('Contraseña')  
15     ).toBeInTheDocument();  
16  
17 });
```

4.5.11. Prueba 11. Envío del formulario de autenticación

Esta prueba de componente valida la interacción del usuario con el formulario de inicio de sesión. Para ello se simula el ingreso de las credenciales en los campos correspondientes y posteriormente se ejecuta el evento asociado al botón de autenticación. Finalmente, se verifica que la función encargada del proceso de inicio de sesión sea invocada con los parámetros ingresados por el usuario.

```
1  it('should call login on submit', async () => {
2
3      render(<LoginPage />);
4
5      const user = screen.getByLabelText('Usuario');
6      const pass = screen.getByLabelText('Contraseña');
7      const btn = screen.getByRole('button', {
8          name: /Iniciar sesión/i
9      });
10
11     fireEvent.change(user, {
12         target: { value: 'admin' }
13     });
14
15     fireEvent.change(pass, {
16         target: { value: '123456' }
17     });
18
19     fireEvent.click(btn);
20
21     await waitFor(() => {
22
23         expect(mockLogin)
24             .toHaveBeenCalledWith(
25                 'admin',
26                 '123456'
27             );
28
29     });
30
31 });
```

La ejecución satisfactoria de esta prueba confirma que el formulario captura correctamente las credenciales proporcionadas por el usuario y las envía al servicio de autenticación sin alterar la información ingresada.

4.5.12. Prueba 12. Visualización de mensajes de error durante la autenticación

Esta prueba verifica el comportamiento de la interfaz cuando el proceso de autenticación falla. Se simula una respuesta negativa del servicio de inicio de sesión mediante un objeto de error y posteriormente se comprueba que dicho mensaje sea mostrado correctamente al usuario, proporcionando una retroalimentación clara sobre el motivo del fallo.

```
1  it('should show error if login fails', async () => {
2
3      mockLogin.mockRejectedValueOnce({
4          message: 'Credenciales inválidas'
5      });
6
7      render(<LoginPage />);
8
9      const btn = screen.getByRole('button', {
10         name: /Iniciar sesión/i
11     });
12
13     fireEvent.click(btn);
14
15     expect(
16         await screen.findByText(
17             'Credenciales inválidas'
18         )
19     ).toBeInTheDocument();
20
21 });
```

La correcta ejecución de esta prueba garantiza que la interfaz informa oportunamente al usuario cuando las credenciales suministradas no son válidas, mejorando la experiencia de uso y permitiendo identificar el motivo del rechazo de la autenticación.

4.6. Resultados

Una vez ejecutados todos los casos de prueba definidos para el requerimiento **REQ001 - Inicio de Sesión**, se obtuvo un total de **12 pruebas ejecutadas**, de las cuales **12 finalizaron exitosamente** y **ninguna presentó fallos**. Esto representa un porcentaje de éxito del **100 %**, evidenciando que el módulo de autenticación cumple satisfactoriamente con los requisitos funcionales establecidos.

Las pruebas realizadas permitieron validar el correcto funcionamiento del proceso de autenticación de usuarios, la validación de credenciales, el manejo de errores ante accesos inválidos, la actualización segura de contraseñas, el desbloqueo automático de cuentas cuando expira el período de bloqueo y el comportamiento de la interfaz gráfica durante las diferentes interacciones del usuario.

Asimismo, los resultados obtenidos demuestran que existe una correcta integración entre los componentes del frontend y los servicios del backend, garantizando que las solicitudes enviadas desde la interfaz sean procesadas adecuadamente por la lógica de negocio y que las respuestas generadas sean reflejadas correctamente en la aplicación.

En conjunto, las evidencias obtenidas durante la ejecución de las pruebas confirman que el requerimiento **REQ001** satisface los criterios de aceptación definidos para el módulo de inicio de sesión y proporciona una base sólida para el funcionamiento seguro del resto de funcionalidades del sistema.

5. REQ002 - Recuperación de Contraseña

5.1. Descripción

El requerimiento **REQ002** corresponde al módulo de recuperación y actualización de contraseña del sistema. Su objetivo es garantizar que los usuarios puedan modificar sus credenciales de acceso de manera segura, validando previamente la información ingresada y mostrando mensajes apropiados ante escenarios de éxito o error.

Las pruebas realizadas se enfocan en el comportamiento del componente de interfaz de usuario encargado del cambio de contraseña, verificando tanto las validaciones del formulario como la comunicación con el servicio responsable de actualizar las credenciales.

5.2. Archivos involucrados

Archivo	Cantidad de pruebas
LoginPage.test.jsx	4
Total	4

Cuadro 5: Distribución de casos de prueba del requerimiento REQ002.

5.3. Casos de prueba ejecutados

5.3.1. Frontend

N°	Caso de prueba	Estado
1	should render change password panel	Aprobado
2	should show error if new passwords do not match	Aprobado
3	should call changePassword and succeed	Aprobado
4	should handle changePassword error	Aprobado

5.4. Resumen de ejecución

Total de pruebas ejecutadas	4
Pruebas aprobadas	4
Pruebas no aprobadas	0
Porcentaje de éxito	100 %

Cuadro 7: Resultado de la ejecución del requerimiento REQ002.

5.5. Implementación

En esta sección se presentan las pruebas automatizadas implementadas para validar el funcionamiento del proceso de recuperación y actualización de contraseña desde la interfaz de usuario.

5.5.1. Prueba 1. Visualización del panel de cambio de contraseña

Esta prueba verifica que el componente de autenticación despliegue correctamente el formulario destinado al cambio de contraseña cuando el usuario selecciona la opción correspondiente. Además, valida que los campos necesarios para completar el proceso sean visibles dentro de la interfaz.

```
1  it('should render change password panel', async () => {
2
3      render(<LoginPage />);
4
5      const changePwdBtn = screen.getByText('Cambiar contraseña');
6      fireEvent.click(changePwdBtn);
7
8      expect(
9          await screen.findByText('Contraseña actual')
10     ).toBeInTheDocument();
11
12     expect(
13         screen.getByText('Confirmar nueva contraseña')
14     ).toBeInTheDocument();
15
16 });
```

La correcta ejecución de esta prueba garantiza que la interfaz habilita el formulario de actualización de contraseña únicamente cuando el usuario lo solicita.

5.5.2. Prueba 2. Validación de coincidencia entre contraseñas nuevas

Esta prueba comprueba que el formulario impida continuar con el proceso cuando la nueva contraseña y su confirmación no coinciden. Se simula el ingreso de valores diferentes en ambos campos y posteriormente se verifica que el sistema muestre el mensaje de validación correspondiente.

```
1  it('should show error if new passwords do not match', async ()
2    => {
3      render(<LoginPage />);
4
5      fireEvent.click(
6        screen.getByText('Cambiar contraseña')
7      );
8
9      const current = screen.getByLabelText('Contraseña actual');
10     const newPwd = screen.getByLabelText('Nueva contraseña');
11     const confirm = screen.getByLabelText('Confirmar nueva
12       contraseña');
13
14     fireEvent.change(current, {
15       target: { value: 'old123' }
16     });
17
18     fireEvent.change(newPwd, {
19       target: { value: 'new1234' }
20     });
21
22     fireEvent.change(confirm, {
23       target: { value: 'different' }
24     });
25
26     fireEvent.click(
27       screen.getByRole('button', {
28         name: /Actualizar contraseña/i
29       })
30     );
31
32     expect(
33       await screen.findByText(
34         /Las contraseñas nuevas no coinciden/i
35       )
36     ).toBeInTheDocument();
37   });
```

La validación de este escenario evita inconsistencias durante la actualización de credenciales y reduce la posibilidad de errores provocados por el ingreso incorrecto de la nueva contraseña.

5.5.3. Prueba 3. Actualización exitosa de contraseña

Esta prueba valida el flujo completo de actualización de contraseña cuando toda la información ingresada por el usuario es correcta. Se simula una respuesta satisfactoria del servicio y posteriormente se verifica que la función encargada del cambio de contraseña sea

invocada con los parámetros esperados y que el sistema notifique el éxito de la operación.

```
1  it('should call changePassword and succeed', async () => {
2
3      mockChangePassword.mockResolvedValueOnce(true);
4
5      render(<LoginPage />);
6
7      fireEvent.click(
8          screen.getByText('Cambiar contraseña')
9      );
10
11     const current = screen.getByLabelText('Contraseña actual');
12     const newPwd = screen.getByLabelText('Nueva contraseña');
13     const confirm = screen.getByLabelText('Confirmar nueva
14         contraseña');
15
16     fireEvent.change(current, {
17         target: { value: 'old123' }
18     });
19
20     fireEvent.change(newPwd, {
21         target: { value: 'new1234' }
22     });
23
24     fireEvent.change(confirm, {
25         target: { value: 'new1234' }
26     });
27
28     fireEvent.click(
29         screen.getByRole('button', {
30             name: /Actualizar contraseña/i
31         })
32     );
33
34     await waitFor(() => {
35         expect(mockChangePassword)
36             .toHaveBeenCalledWith(
37                 'old123',
38                 'new1234'
39             );
40     });
41
42     expect(
43         await screen.findByText(
44             'Contraseña actualizada exitosamente.'
45         )
46     ).toBeInTheDocument();
47 });
```

La ejecución satisfactoria de esta prueba confirma que el formulario envía correcta-

mente la información al servicio responsable del cambio de contraseña y que la interfaz informa al usuario cuando la operación finaliza exitosamente.

5.5.4. Prueba 4. Manejo de errores durante el cambio de contraseña

Esta prueba verifica la respuesta de la interfaz cuando el servicio encargado de actualizar la contraseña devuelve un error. Para ello se simula una excepción indicando que la contraseña actual es incorrecta y posteriormente se comprueba que dicho mensaje sea mostrado al usuario.

```
1  it('should handle changePassword error', async () => {
2
3      mockChangePassword.mockRejectedValueOnce(
4          new Error('Old password incorrect')
5      );
6
7      render(<LoginPage />);
8
9      fireEvent.click(
10         screen.getByText('Cambiar contraseña')
11     );
12
13     const current = screen.getByLabelText('Contraseña actual');
14     const newPwd = screen.getByLabelText('Nueva contraseña');
15     const confirm = screen.getByLabelText('Confirmar nueva
16         contraseña');
17
18     fireEvent.change(current, {
19         target: { value: 'old123' }
20     });
21
22     fireEvent.change(newPwd, {
23         target: { value: 'new1234' }
24     });
25
26     fireEvent.change(confirm, {
27         target: { value: 'new1234' }
28     });
29
30     fireEvent.click(
31         screen.getByRole('button', {
32             name: /Actualizar contraseña/i
33         })
34     );
35
36     expect(
37         await screen.findByText(
38             /Old password incorrect/i
39         )
40     ).toBeInTheDocument();
41 });
```

Esta prueba garantiza que el sistema proporciona retroalimentación clara al usuario cuando ocurre un error durante el proceso de actualización de credenciales, facilitando la identificación del problema y evitando comportamientos inesperados en la interfaz.

5.6. Resultados

La ejecución de los casos de prueba definidos para el requerimiento **REQ002 - Recuperación de Contraseña** permitió validar satisfactoriamente el funcionamiento del proceso de actualización de credenciales implementado en la interfaz de usuario. Se ejecutaron un total de **4 casos de prueba**, obteniendo **4 pruebas aprobadas** y **0 pruebas no aprobadas**, lo que representa un porcentaje de éxito del **100 %**.

Los resultados obtenidos demuestran que el sistema presenta correctamente el formulario de cambio de contraseña, valida la consistencia de los datos ingresados por el usuario, ejecuta correctamente la actualización de las credenciales cuando la información es válida y comunica adecuadamente tanto los escenarios de éxito como los errores generados durante el proceso.

6. REQ003 - Gestión de Pacientes

6.1. Descripción

El requerimiento **REQ003** corresponde al módulo de gestión de pacientes del sistema, responsable del registro, consulta, actualización y eliminación de la información clínica de los pacientes. Este módulo constituye uno de los componentes principales de la aplicación, ya que centraliza los datos personales y médicos utilizados posteriormente durante el proceso de agendamiento de citas y atención clínica.

Para validar este requerimiento se ejecutaron pruebas unitarias, de integración, sistema, aceptación, cobertura y componente, verificando tanto la lógica de negocio implementada en el backend como el comportamiento de la interfaz gráfica y de los servicios consumidos desde el frontend.

6.2. Archivos involucrados

Archivo	Cantidad de pruebas
01-unitarias.test.js	8
02-integracion.test.js	2
03-sistema.test.js	1
04-aceptacion.test.js	2
05-cobertura.test.js	5
api.service.test.js	1
PatientForm.test.jsx	5
Patients.test.jsx	8
Total	32

Cuadro 8: Distribución de casos de prueba del requerimiento REQ003.

6.3. Casos de prueba ejecutados

6.3.1. Pruebas Unitarias

N°	Caso de prueba	Estado
----	----------------	--------

1	Debe crear un paciente con todos los inputs válidos	Aprobado
2	Debe fallar si falta el campo nombre	Aprobado
3	Debe fallar si la edad es negativa	Aprobado
4	Debe fallar si la edad es mayor a 150	Aprobado
5	Debe fallar si el peso es cero o negativo	Aprobado
6	Debe fallar si la estatura es cero o negativa	Aprobado
7	Debe crear un paciente aunque falten campos opcionales	Aprobado
8	Debe fallar al buscar un paciente por un identificador inexistente	Aprobado

6.4. Implementación

A continuación se presentan las pruebas unitarias implementadas para validar las reglas de negocio del modelo de pacientes.

6.4.1. Prueba 1. Registro de un paciente con información válida

Esta prueba verifica que el modelo de pacientes permita registrar correctamente un paciente cuando todos los datos obligatorios cumplen las reglas de validación definidas por la aplicación. Posteriormente se comprueba que la información almacenada corresponda con los valores enviados durante la creación del registro.

```
1  it('1. Debe crear un paciente con todos los inputs válidos',
2    async () => {
3
4      const p = await PatientModel.create(validData);
5
6      expect(p.nombre).toBe('Juan');
7      expect(p.edad).toBe(30);
8    });
```

La ejecución satisfactoria de esta prueba confirma que el modelo acepta información válida y persiste correctamente los datos del paciente.

6.4.2. Prueba 2. Validación del campo obligatorio nombre

Esta prueba comprueba que el sistema rechace la creación de un paciente cuando el campo **nombre** no contiene información. Se espera que el modelo genere una excepción indicando el incumplimiento de las restricciones definidas para dicho atributo.

```
1  it('2. Debe fallar si falta el campo "nombre"', async () => {
2
3      await expect(
4          PatientModel.create({
5              ...validData,
6              nombre: ''
7          })
8      ).rejects.toThrow(/nombre/);
9
10 });
```

La correcta ejecución de esta prueba garantiza la obligatoriedad del nombre dentro del registro de pacientes.

6.4.3. Prueba 3. Validación de edad negativa

Esta prueba verifica que el modelo impida registrar pacientes cuya edad sea inferior a cero, evitando el almacenamiento de información inconsistente dentro de la base de datos.

```
1  it('4. Debe fallar si la edad es negativa', async () => {
2
3      await expect(
4          PatientModel.create({
5              ...validData,
6              edad: -5
7          })
8      ).rejects.toThrow(/Edad inválida/);
9
10 });
```

La validación de este escenario garantiza que únicamente puedan registrarse edades dentro del rango permitido.

6.4.4. Prueba 4. Validación de edad máxima permitida

Esta prueba valida que el sistema rechace edades superiores al límite establecido por las reglas de negocio. Para ello se intenta registrar un paciente con una edad de 200 años y posteriormente se verifica que la operación sea rechazada.

```
1  it('5. Debe fallar si la edad es mayor a 150', async () => {
2
3      await expect(
4          PatientModel.create({
5              ...validData,
6              edad: 200
7          })
8      ).rejects.toThrow(/Edad inválida/);
9
10 });
```

Esta validación evita el registro de valores fuera del rango permitido para la edad del paciente.

6.4.5. Prueba 5. Validación del peso corporal

Esta prueba verifica que el sistema impida registrar pacientes cuyo peso sea igual o inferior a cero kilogramos, garantizando la coherencia de la información médica almacenada.

```
1  it('6. Debe fallar si el peso es cero o negativo', async () => {
2
3      await expect(
4          PatientModel.create({
5              ...validData,
6              peso: 0
7          })
8      ).rejects.toThrow(/Peso inválido/);
9
10 });
```

La ejecución correcta de esta prueba asegura que el atributo correspondiente al peso cumpla las restricciones establecidas por el modelo.

6.4.6. Prueba 6. Validación de la estatura

Esta prueba comprueba que el sistema rechace registros cuya estatura sea igual o inferior a cero centímetros, evitando inconsistencias dentro de la información clínica del paciente.

```
1  it('7. Debe fallar si la estatura es cero o negativa', async ()  
    => {  
2  
3      await expect(  
4          PatientModel.create({  
5              ...validData,  
6              estatura: -10  
7          })  
8      ).rejects.toThrow(/Estatura inválida/);  
9  
10 });
```

Esta validación garantiza la integridad de los datos biométricos registrados para cada paciente.

6.4.7. Prueba 7. Registro con campos opcionales vacíos

Esta prueba verifica que el modelo permita crear correctamente un paciente aun cuando determinados campos opcionales, como el historial médico y el número telefónico, no sean proporcionados durante el registro.

```
1  it('8. Debe crear un paciente aunque falten inputs opcionales (  
    historialMedico, telefono)', async () => {  
2  
3      const { historialMedico, telefono, ...requiredData } =  
        validData;  
4  
5      const p = await PatientModel.create(requiredData);  
6  
7      expect(p.historialMedico).toBe('');  
8      expect(p.telefono).toBe('');  
9  
10 });
```

La ejecución satisfactoria confirma que únicamente los campos obligatorios son requeridos para registrar un nuevo paciente.

6.4.8. Prueba 8. Búsqueda de un paciente inexistente

Esta prueba valida el comportamiento del sistema cuando se intenta recuperar un paciente utilizando un identificador que no existe dentro de la base de datos. El resultado esperado consiste en generar una excepción indicando que el registro solicitado no fue encontrado.

```
1  it('10. Debe fallar al buscar paciente por un ID inexistente',  
2    async () => {  
3      
4      await expect(  
5        PatientModel.getById(  
6          '60c72b2f9b1d4c3d8c1e4d5b'  
7        )  
8      ).rejects.toThrow(/no encontrado/);  
9  });
```

Esta prueba garantiza que el sistema gestione correctamente las búsquedas de registros inexistentes y proporcione una respuesta consistente ante este tipo de situaciones.

6.4.9. Prueba 9. Consulta del listado de pacientes

Esta prueba de integración verifica el comportamiento del servicio encargado de recuperar la información de los pacientes registrados en el sistema. Debido a que el acceso puede depender del mecanismo de autenticación configurado durante las pruebas, el caso contempla como válidas las respuestas correspondientes a una consulta exitosa (**HTTP 200**) o aquellas que indiquen la necesidad de autenticación (**HTTP 401** o **HTTP 403**).

El objetivo principal consiste en comprobar que el endpoint responda de forma controlada y consistente independientemente del contexto de autenticación.

```
1  it('19. GET /api/patients - Debe requerir autenticación o  
2    devolver arreglo', async () => {  
3      
4      const res = await request(app)  
5        .get('/api/patients');  
6      
7      expect([200,401,403]).toContain(res.status);  
8  });
```

La ejecución satisfactoria de esta prueba confirma que el servicio de consulta de pacientes responde adecuadamente ante solicitudes realizadas con y sin autenticación, respetando las políticas de seguridad implementadas por la aplicación.

6.4.10. Prueba 10. Validación de datos obligatorios durante el registro de pacientes

Esta prueba de integración verifica que el endpoint encargado del registro de pacientes valide correctamente la presencia de la información obligatoria antes de intentar procesar la solicitud. Para ello se envía una petición HTTP con un cuerpo vacío y posteriormente se comprueba que el servidor rechace la operación devolviendo el código de estado **400 (Bad Request)**.


```
1 it('21. POST /api/patients - Debe fallar validacion de integraci  
   ón si los inputs están vacíos', async () => {  
2  
3     const res = await request(app)  
4       .post('/api/patients')  
5       .send({});  
6  
7     expect(res.status).toBe(400);  
8  
9   });
```

Esta prueba garantiza que el sistema no permita registrar pacientes cuando no se proporciona la información mínima requerida, evitando el almacenamiento de registros incompletos y preservando la integridad de los datos almacenados.

6.4.11. Prueba 11. Flujo completo de registro de paciente y agendamiento de cita

Esta prueba de sistema valida uno de los principales escenarios funcionales de la aplicación. El flujo inicia con el registro de un nuevo paciente, continúa con la obtención del identificador generado por el sistema y finaliza con la programación de una cita médica para dicho paciente. El objetivo consiste en comprobar la correcta integración entre los módulos de gestión de pacientes y gestión de citas, verificando además que la cita creada adopte inicialmente el estado **pendiente**.

```
1  it('24. Flujo Completo: Admin crea un paciente y luego agenda
    una cita exitosamente', async () => {
2
3      const doctor = await Doctor.create({
4          name: 'Dr. House',
5          specialty: 'General'
6      });
7
8      const patientRes = await request(app)
9          .post('/api/patients')
10         .send({
11             nombre: 'Test',
12             apellido: 'Sistema',
13             edad: 30,
14             peso: 70,
15             estatura: 170,
16             sexo: 'Hombre',
17             ocupacion: 'QA'
18         });
19
20     const patientId = patientRes.body.data._id;
21
22     const apptRes = await request(app)
23         .post('/api/appointments')
24         .send({
25             patientId: patientId,
26             doctorId: doctor._id,
27             fecha: '2030-01-01',
28             hora: '10:00'
29         });
30
31     expect(apptRes.status).toBe(201);
32     expect(apptRes.body.data.estado).toBe('pendiente');
33
34 });
```

La correcta ejecución de esta prueba confirma que el sistema integra satisfactoriamente el registro de pacientes con el proceso de agendamiento de citas, permitiendo que la información generada por un módulo sea utilizada correctamente por el siguiente dentro del flujo de atención médica.

6.4.12. Prueba 12. Agendamiento exitoso de una cita en un horario disponible

Esta prueba de aceptación valida uno de los escenarios funcionales esperados por el usuario final. Se registra previamente un médico y un paciente, para posteriormente solicitar la creación de una cita médica en un horario libre. Finalmente se verifica que la cita sea registrada correctamente y que su estado inicial corresponda a **pendiente**, indicando que el proceso de programación fue exitoso.

```
1  it('27. Dado un paciente nuevo, Cuando solicita agendar cita en
   horario libre, Entonces se programa como "pendiente"', async
   () => {
2
3     const doctor = await Doctor.create({
4       name: 'Dr. Test',
5       specialty: 'General'
6     });
7
8     const patient = await Patient.create({
9       nombre: 'A',
10      apellido: 'B',
11      edad: 20,
12      peso: 50,
13      estatura: 150,
14      sexo: 'Mujer',
15      ocupacion: 'Estudiante'
16    });
17
18    const appt = await AppointmentModel.create({
19      patientId: patient._id,
20      doctorId: doctor._id,
21      fecha: '2030-01-01',
22      hora: '10:00'
23    });
24
25    expect(appt.estado).toBe('pendiente');
26
27  });
```

Esta prueba demuestra que el sistema cumple con uno de los principales criterios de aceptación del módulo, permitiendo que un paciente pueda agendar exitosamente una cita cuando el horario seleccionado se encuentra disponible.

6.4.13. Prueba 13. Actualización de antecedentes médicos de un paciente

Esta prueba de aceptación verifica que un paciente registrado pueda actualizar correctamente su historial médico. Para ello se crea inicialmente un registro con un antecedente básico y posteriormente se ejecuta el proceso de actualización mediante el modelo de pacientes. Finalmente se comprueba que la información almacenada corresponda con el nuevo valor ingresado.

```
1  it('28. Dado un paciente existente, Cuando actualiza sus
   antecedentes médicos, Entonces los cambios se reflejan',
   async () => {
2
3     const p = await Patient.create({
4       nombre: 'A',
5       apellido: 'B',
6       edad: 20,
7       peso: 50,
8       estatura: 150,
9       sexo: 'Mujer',
10      ocupacion: 'Estudiante',
11      historialMedico: 'Ninguno'
12    });
13
14    const res = await PatientModel.update(
15      p._id,
16      {
17        historialMedico: 'Alergia al polen'
18      }
19    );
20
21    expect(res.historialMedico)
22      .toBe('Alergia al polen');
23
24  });
```

La ejecución satisfactoria de esta prueba garantiza que el módulo de gestión de pacientes conserva la consistencia de la información clínica y permite actualizar correctamente los antecedentes médicos sin afectar el resto de los datos asociados al paciente.

6.4.14. Prueba 14. Obtención de la lista de pacientes

Esta prueba de cobertura verifica el funcionamiento del controlador encargado de recuperar todos los pacientes registrados en el sistema. Previamente se inserta un paciente de prueba en la base de datos y posteriormente se realiza una petición HTTP al endpoint correspondiente. Finalmente se comprueba que la respuesta sea satisfactoria y que el listado obtenido contenga exactamente un registro.

```
1  it('GET /api/patients - Debe obtener la lista de pacientes',  
    async () => {  
2  
3      const res = await request(app)  
4          .get('/api/patients');  
5  
6      expect(res.status).toBe(200);  
7      expect(res.body.data.length).toBe(1);  
8  
9  });
```

La correcta ejecución de esta prueba confirma que el controlador consulta correctamente la información almacenada y devuelve los pacientes registrados mediante una respuesta HTTP satisfactoria.

6.4.15. Prueba 15. Consulta de un paciente mediante su identificador

Esta prueba valida que el controlador permita recuperar la información completa de un paciente utilizando su identificador único. Después de registrar previamente un paciente, se realiza una consulta utilizando su ID y se verifica que los datos obtenidos correspondan al registro almacenado.

```
1  it('GET /api/patients/:id - Debe obtener un paciente por id',  
    async () => {  
2  
3      const res = await request(app)  
4          .get('/api/patients/${patientId}');  
5  
6      expect(res.status).toBe(200);  
7      expect(res.body.data.nombre).toBe('Cov');  
8  
9  });
```

La prueba garantiza que el servicio de consulta individual devuelve correctamente la información asociada al paciente solicitado cuando el identificador existe dentro de la base de datos.

6.4.16. Prueba 16. Actualización de la información de un paciente

Esta prueba de cobertura verifica el funcionamiento del proceso de actualización de pacientes. Para ello se modifica el número telefónico de un paciente previamente registrado y posteriormente se comprueba que la información almacenada haya sido actualizada correctamente.

```
1  it('PUT /api/patients/:id - Debe actualizar paciente', async ()
    => {
2
3      const res = await request(app)
4          .put('/api/patients/${patientId}')
5          .send({
6              telefono: '555555'
7          });
8
9      expect(res.status).toBe(200);
10     expect(res.body.data.telefono).toBe('555555');
11
12 });
```

La ejecución satisfactoria de esta prueba confirma que el controlador procesa correctamente las solicitudes de actualización y persiste los cambios realizados sobre la información del paciente.

6.4.17. Prueba 17. Eliminación de un paciente

Esta prueba verifica el funcionamiento del proceso de eliminación de pacientes. Después de enviar la solicitud de eliminación, se consulta directamente la base de datos para comprobar que el registro ya no existe.

```
1  it('DELETE /api/patients/:id - Debe eliminar paciente', async ()
    => {
2
3      const res = await request(app)
4          .delete('/api/patients/${patientId}');
5
6      expect(res.status).toBe(200);
7
8      const verify = await Patient.findById(patientId);
9
10     expect(verify).toBeNull();
11
12 });
```

Esta prueba garantiza que el controlador elimina correctamente el registro solicitado y que dicho paciente deja de existir dentro de la base de datos.

6.4.18. Prueba 18. Manejo de errores internos durante la consulta de pacientes

Esta prueba de cobertura evalúa la capacidad del controlador para gestionar errores inesperados provenientes de la capa de acceso a datos. Para ello se simula una excepción en el método encargado de recuperar los pacientes mediante el uso de un *mock*. Posteriormente se verifica que el controlador responda con el código HTTP correspondiente al

error interno del servidor.

```
1  it('GET /api/patients - Debe devolver 500 si PatientModel falla',
2    , async () => {
3      jest.spyOn(PatientModel, 'getAll')
4        .mockRejectedValueOnce(
5          new Error('DB Error')
6        );
7
8      const res = await request(app)
9        .get('/api/patients');
10
11      expect(res.status).toBe(500);
12      expect(res.body.message).toBe('DB Error');
13
14    });
```

La correcta ejecución de esta prueba demuestra que el sistema implementa un adecuado manejo de excepciones, evitando fallos no controlados y proporcionando una respuesta consistente cuando ocurre un error durante el acceso a la base de datos.

6.4.19. Prueba 19. Validación de los servicios de comunicación para la gestión de pacientes

Esta prueba verifica el correcto funcionamiento de la capa de servicios encargada de la comunicación entre la interfaz de usuario y la API REST del sistema. Para ello se utiliza un *mock* de la función `fetch`, simulando respuestas exitosas del servidor y comprobando que cada operación del servicio de pacientes invoque el endpoint correspondiente utilizando el método HTTP adecuado.

Además de validar las operaciones sobre pacientes, el archivo también comprueba el funcionamiento básico de los servicios de citas y doctores, garantizando el correcto acceso a los recursos principales del sistema desde el cliente.

```
1  it('Patient Service Methods', async () => {
2
3      mockSuccess([]);
4
5      await patientService.getAll();
6
7      expect(global.fetch).toHaveBeenCalledWith(
8          expect.stringContaining('/patients'),
9          expect.objectContaining({
10              method: 'GET'
11          })
12      );
13
14      mockSuccess({});
15
16      await patientService.getById('1');
17
18      expect(global.fetch).toHaveBeenCalledWith(
19          expect.stringContaining('/patients/1'),
20          expect.objectContaining({
21              method: 'GET'
22          })
23      );
24
25      mockSuccess({});
26
27      await patientService.create({
28          name: 'test'
29      });
30
31      expect(global.fetch).toHaveBeenCalledWith(
32          expect.stringContaining('/patients'),
33          expect.objectContaining({
34              method: 'POST'
35          })
36      );
37
38      mockSuccess({});
39
40      await patientService.update(
41          '1',
42          {
43              name: 'test2'
44          }
45      );
46
47      expect(global.fetch).toHaveBeenCalledWith(
48          expect.stringContaining('/patients/1'),
49          expect.objectContaining({
50              method: 'PUT'
51          })
52      );
53
54      mockSuccess({});
55
56      await patientService.delete('1');
```


La correcta ejecución de esta prueba confirma que la capa de servicios del frontend establece correctamente la comunicación con la API REST para realizar las operaciones de consulta, registro, actualización y eliminación de pacientes. Asimismo, la utilización de objetos simulados (*mocks*) permite verificar el comportamiento de los servicios de forma aislada, sin depender de la disponibilidad del servidor o de la base de datos durante la ejecución de las pruebas.

6.4.20. Prueba 20. Verificación del renderizado del formulario de pacientes

Esta prueba de componente verifica que la interfaz gráfica del formulario de registro de pacientes se renderice correctamente. Para ello se comprueba la presencia de todos los controles necesarios para el ingreso de información personal y clínica del paciente, garantizando que el usuario disponga de todos los campos requeridos antes de iniciar el proceso de registro.

```
1  it('renders form elements correctly', () => {
2
3      render(<PatientForm />);
4
5      expect(screen.getByText('Datos personales'))
6          .toBeInTheDocument();
7
8      expect(screen.getByLabelText('Nombre *'))
9          .toBeInTheDocument();
10
11     expect(screen.getByLabelText('Apellido *'))
12         .toBeInTheDocument();
13
14     expect(screen.getByLabelText('Edad *'))
15         .toBeInTheDocument();
16
17     expect(screen.getByLabelText('Sexo *'))
18         .toBeInTheDocument();
19
20     expect(screen.getByLabelText('Ocupación *'))
21         .toBeInTheDocument();
22
23     expect(screen.getByLabelText('Peso (kg) *'))
24         .toBeInTheDocument();
25
26     expect(screen.getByLabelText('Estatura (cm) *'))
27         .toBeInTheDocument();
28
29 });
```

La correcta ejecución de esta prueba garantiza que el formulario presenta todos los controles necesarios para registrar la información del paciente, permitiendo una interacción adecuada desde el primer momento.

6.4.21. Prueba 21. Validación de campos obligatorios

Esta prueba verifica que el formulario aplique correctamente las reglas de validación antes de permitir el registro de un paciente. Se intenta enviar el formulario sin ingresar información, comprobando posteriormente que el sistema muestre los mensajes de error correspondientes para cada uno de los campos obligatorios.

```
1  it('shows validation errors for empty/invalid fields', () => {
2
3      render(<PatientForm />);
4
5      fireEvent.click(
6          screen.getByRole('button',
7              { name: /Registrar paciente/i })
8      );
9
10     expect(screen.getByText('El nombre es requerido'))
11         .toBeInTheDocument();
12
13     expect(screen.getByText('El apellido es requerido'))
14         .toBeInTheDocument();
15
16     expect(screen.getByText('Ingresa una edad válida (0-150)'))
17         .toBeInTheDocument();
18
19     expect(screen.getByText('Ingresa un peso válido en kg'))
20         .toBeInTheDocument();
21
22     expect(screen.getByText('Ingresa una estatura válida en cm'))
23         .toBeInTheDocument();
24
25     expect(screen.getByText('Selecciona el sexo'))
26         .toBeInTheDocument();
27
28     expect(screen.getByText('La ocupación es requerida'))
29         .toBeInTheDocument();
30
31 });
```

La prueba confirma que el formulario implementa correctamente las validaciones de entrada, evitando el envío de información incompleta hacia el servidor.

6.4.22. Prueba 22. Cálculo automático del índice de masa corporal

Esta prueba valida el comportamiento dinámico del formulario al calcular automáticamente el índice de masa corporal (IMC) a partir del peso y la estatura ingresados por el usuario. El cálculo se realiza inmediatamente después de modificar ambos valores, verificando posteriormente que el resultado mostrado corresponda al valor esperado.

```
1  it('calculates IMC correctly when peso and estatura are provided', () => {
2
3      render(<PatientForm />);
4
5      const pesoInput =
6          screen.getByLabelText('Peso (kg) *');
7
8      const estaturaInput =
9          screen.getByLabelText('Estatura (cm) *');
10
11     fireEvent.change(pesoInput, {
12         target: {
13             name: 'peso',
14             value: '70'
15         }
16     });
17
18     fireEvent.change(estaturaInput, {
19         target: {
20             name: 'estatura',
21             value: '175'
22         }
23     });
24
25     expect(screen.getByText('22.9'))
26         .toBeInTheDocument();
27
28 });
```

La correcta ejecución de esta prueba demuestra que el formulario incorpora correctamente la lógica de cálculo automático del IMC, proporcionando al usuario información clínica adicional durante el registro.

6.4.23. Prueba 23. Registro exitoso de un paciente

Esta prueba verifica el comportamiento del formulario cuando todos los datos ingresados son válidos. Se simula el llenado completo del formulario y posteriormente se comprueba que el servicio encargado del registro sea invocado correctamente. Finalmente se valida que se ejecuten las funciones asociadas al cierre del formulario y a la notificación de éxito.

```
1  it('calls createPatient and onSuccess on valid submit', async ()
2    => {
3      mockCreatePatient
4        .mockResolvedValueOnce({_id: '123'});
5
6      const onSuccess = vi.fn();
7      const onClose = vi.fn();
8
9      render(
10        <PatientForm
11          onSuccess={onSuccess}
12          onClose={onClose}
13        />
14      );
15
16      ...
17
18      fireEvent.click(
19        screen.getByRole('button',
20          {name:/Registrar paciente/i})
21      );
22
23      await waitFor(()=>{
24
25        expect(mockCreatePatient)
26          .toHaveBeenCalledWith(
27            expect.objectContaining({
28              nombre: 'Ana',
29              apellido: 'Lopez',
30              edad: 25,
31              sexo: 'Femenino',
32              peso: 60,
33              estatura: 160
34            })
35          );
36
37        expect(onSuccess).toHaveBeenCalled();
38
39        expect(onClose).toHaveBeenCalled();
40
41      });
42    });
43  });
```

La prueba confirma que el formulario recopila correctamente la información ingresada por el usuario, invoca el servicio de registro y ejecuta las acciones posteriores cuando la operación finaliza exitosamente.

6.4.24. Prueba 24. Manejo de errores durante el registro

Esta prueba evalúa el comportamiento del formulario cuando ocurre una excepción durante el proceso de registro del paciente. Para ello se simula un error proveniente del servicio de creación y posteriormente se verifica que el mensaje correspondiente sea presentado al usuario dentro de la interfaz.

```
1  it('shows generic error if createPatient throws', async () => {
2
3      mockCreatePatient
4          .mockRejectedValueOnce(
5              new Error('Server error')
6          );
7
8      render(<PatientForm />);
9
10     ...
11
12     fireEvent.click(
13         screen.getByRole('button',
14             {name:/Registrar paciente/i})
15     );
16
17     await waitFor(()=>{
18
19         expect(
20             screen.getByText('Server error')
21         ).toBeInTheDocument();
22
23     });
24
25 });
```

La correcta ejecución de esta prueba garantiza que el formulario responde adecuadamente ante errores producidos durante la comunicación con el servidor, proporcionando retroalimentación inmediata al usuario y evitando fallos silenciosos dentro de la aplicación.

6.4.25. Prueba 25. Visualización del listado de pacientes

Esta prueba verifica que la página de gestión de pacientes muestre correctamente la información almacenada en el contexto de la aplicación. Se renderiza el componente utilizando un proveedor de rutas y posteriormente se valida que los datos principales del paciente sean visibles dentro de la tabla.

```
1  it('should render patients table', () => {
2
3      render(
4          <BrowserRouter>
5              <Patients />
6          </BrowserRouter>
7      );
8
9      expect(screen.getAllByText(/Juan Perez/)[0])
10         .toBeInTheDocument();
11
12      expect(screen.getAllByText(/30 años/)[0])
13         .toBeInTheDocument();
14
15      expect(screen.getAllByText(/Ingeniero/)[0])
16         .toBeInTheDocument();
17
18  });
```

La correcta ejecución de esta prueba garantiza que el componente presenta correctamente la información de los pacientes recuperada desde el contexto de la aplicación.

6.4.26. Prueba 26. Búsqueda de pacientes

Esta prueba valida el funcionamiento del buscador incorporado en la página de pacientes. Se simulan diferentes criterios de búsqueda verificando que el sistema filtre correctamente los registros existentes y oculte aquellos que no cumplen con el criterio ingresado.

```
1  it('should search patients', () => {
2
3      render(
4          <BrowserRouter>
5              <Patients />
6          </BrowserRouter>
7      );
8
9      const searchInput =
10         screen.getByPlaceholderText(
11             /Buscar por nombre/i
12         );
13
14         fireEvent.change(searchInput,{
15             target:{value:'Ingeniero'}
16         });
17
18         expect(screen.getAllByText(/Juan Perez/)[0])
19             .toBeInTheDocument();
20
21         fireEvent.change(searchInput,{
22             target:{value:'Inexistente'}
23         });
24
25         expect(screen.queryByText(/Juan Perez/))
26             .not.toBeInTheDocument();
27
28     });
```

La prueba confirma que el mecanismo de búsqueda filtra dinámicamente la información mostrada al usuario conforme al texto ingresado.

6.4.27. Prueba 27. Apertura del formulario de registro

Esta prueba verifica que el usuario pueda abrir el formulario destinado al registro de nuevos pacientes mediante el botón correspondiente dentro de la interfaz.

```
1  it('should open modal to register patient', () => {
2
3      render(
4          <BrowserRouter>
5              <Patients />
6          </BrowserRouter>
7      );
8
9      const btn =
10         screen.getByText('+ Registrar paciente');
11
12         fireEvent.click(btn);
13
14         expect(
15             screen.getByText(
16                 'Registrar nuevo paciente'
17             )
18         ).toBeInTheDocument();
19
20     });
```

La correcta ejecución de esta prueba demuestra que la interfaz permite iniciar correctamente el proceso de registro de un nuevo paciente.

6.4.28. Prueba 28. Eliminación de un paciente

Esta prueba verifica el flujo de eliminación de pacientes. Se simula la confirmación del usuario mediante el cuadro de diálogo correspondiente y posteriormente se valida que el servicio encargado de eliminar el registro sea invocado con el identificador correcto.


```
1  it('should call deletePatient on delete button click', async ()
2    => {
3      render(
4        <BrowserRouter>
5          <Patients />
6        </BrowserRouter>
7      );
8
9      const delBtn =
10        screen.getByText('X');
11
12      fireEvent.click(delBtn);
13
14      expect(window.confirm)
15        .toHaveBeenCalledWith(
16        '¿Eliminar al paciente "Juan Perez"?'
17      );
18
19      await waitFor(()=>{
20
21        expect(mockDeletePatient)
22          .toHaveBeenCalledWith('1');
23
24      });
25
26  });
```

La prueba garantiza que la eliminación de pacientes requiere confirmación previa y ejecuta correctamente el servicio correspondiente.

6.4.29. Prueba 29. Visualización del detalle de un paciente

Esta prueba verifica que el sistema permita consultar la información detallada de un paciente mediante una ventana modal, comprobando además que dicha ventana pueda cerrarse correctamente.

```
1  it('should open patient details modal', () => {
2
3      render(
4          <BrowserRouter>
5              <Patients />
6          </BrowserRouter>
7      );
8
9      fireEvent.click(
10         screen.getByText('Ver')
11     );
12
13     expect(
14         screen.getByText(
15             'Detalle del paciente'
16         )
17     ).toBeInTheDocument();
18
19     expect(
20         screen.getAllByText(
21             /Sin alergias/
22         )[0]
23     ).toBeInTheDocument();
24
25     const cerrarBtn =
26         screen.getAllByRole(
27             'button',
28             {name:/Cerrar/i}
29         )[1];
30
31     fireEvent.click(cerrarBtn);
32
33     expect(
34         screen.queryByText(
35             'Detalle del paciente'
36         )
37     ).not.toBeInTheDocument();
38
39 });
```

La correcta ejecución de esta prueba demuestra que el sistema permite consultar y cerrar correctamente la información detallada de un paciente.

6.4.30. Prueba 30. Manejo de errores durante la eliminación

Esta prueba evalúa el comportamiento del componente cuando ocurre un error durante la eliminación de un paciente. Se simula una excepción en el servicio correspondiente verificando que el mensaje sea comunicado al usuario mediante una alerta.

```
1  it('should handle delete patient error', async () => {
2
3      mockDeletePatient
4          .mockRejectedValueOnce(
5              new Error('Delete error')
6          );
7
8      window.alert = vi.fn();
9
10     render(
11         <BrowserRouter>
12             <Patients />
13         </BrowserRouter>
14     );
15
16     fireEvent.click(
17         screen.getByText('X')
18     );
19
20     await waitFor(()=>{
21
22         expect(window.alert)
23             .toHaveBeenCalledWith(
24                 'Delete error'
25             );
26
27     });
28
29 });
```

La prueba confirma que el sistema informa adecuadamente al usuario cuando ocurre un error durante la eliminación de un paciente.

6.4.31. Prueba 31. Cancelación de la eliminación

Esta prueba verifica que el sistema respete la decisión del usuario cuando éste cancela el proceso de eliminación desde el cuadro de confirmación.

```
1  it('should not delete patient if confirm is cancelled', () => {
2
3      window.confirm =
4          vi.fn(() => false);
5
6      render(
7          <BrowserRouter>
8              <Patients />
9          </BrowserRouter>
10     );
11
12     fireEvent.click(
13         screen.getByText('X')
14     );
15
16     expect(mockDeletePatient)
17         .not.toHaveBeenCalled();
18
19 });
```

La correcta ejecución de esta prueba garantiza que ningún registro es eliminado cuando el usuario cancela la operación.

6.4.32. Prueba 32. Cierre del formulario de registro

Esta prueba verifica que el formulario modal utilizado para registrar pacientes pueda cerrarse correctamente desde la interfaz gráfica.

```
1  it('should close patient registration modal', async () => {
2
3      render(
4          <BrowserRouter>
5              <Patients />
6          </BrowserRouter>
7      );
8
9      fireEvent.click(
10         screen.getByText(
11             '+ Registrar paciente'
12         )
13     );
14
15     expect(
16         screen.getByText(
17             'Registrar nuevo paciente'
18         )
19     ).toBeInTheDocument();
20
21     const closeBtns =
22         screen.getAllByRole('button');
23
24     const modalCloseBtn =
25         closeBtns.find(
26             b => b.getAttribute('aria-label')
27                 === 'Cerrar'
28         );
29
30     if(modalCloseBtn){
31
32         fireEvent.click(modalCloseBtn);
33
34         expect(
35             screen.queryByText(
36                 'Registrar nuevo paciente'
37             )
38         ).not.toBeInTheDocument();
39
40     }
41
42 });
```

La prueba confirma que el formulario modal puede cerrarse correctamente sin afectar el funcionamiento general de la página, garantizando una interacción adecuada con la interfaz de usuario.

7. REQ004 - Gestión de Perfiles

7.1. Descripción

El requisito de Gestión de Perfiles comprende las funcionalidades relacionadas con la administración de la información de los doctores disponibles dentro del sistema y las validaciones necesarias para garantizar la consistencia durante la programación de citas. Las pruebas ejecutadas verifican tanto la disponibilidad de los perfiles médicos como las restricciones de negocio asociadas a la asignación de citas, evitando conflictos de horario y validando la correcta recuperación de la información desde los servicios del sistema.

Para validar este requisito se ejecutaron pruebas unitarias, de integración, aceptación, cobertura y pruebas del frontend, obteniéndose un total de nueve casos de prueba ejecutados satisfactoriamente.

Resultado general del requisito

- Total de pruebas ejecutadas: 9
- Pruebas aprobadas: 9
- Pruebas fallidas: 0
- Estado del requisito: Aprobado

7.2. Implementación

7.2.1. Prueba 1. Creación de una cita con paciente y doctor válidos

Esta prueba unitaria verifica que el modelo encargado de administrar las citas permite registrar correctamente una nueva cita cuando tanto el paciente como el doctor existen y la información suministrada cumple todas las reglas de negocio.

```
1  it('11. Debe crear una cita válida con paciente y doctor válidos',
2  ,
3  ,
4  async () => {
5      const cita =
6          await AppointmentModel.create({
7              patientId: testPatient._id,
8              doctorId: testDoctor._id,
9              fecha: '2030-05-15',
10             hora: '10:00'
11         });
12     expect(cita).toBeDefined();
13
14     });
15
16 }
```

La correcta ejecución de esta prueba confirma que el sistema registra adecuadamente nuevas citas cuando todos los datos requeridos son válidos y existen los perfiles asociados.

7.2.2. Prueba 2. Validación de solapamiento de citas

Esta prueba verifica que el sistema impida registrar múltiples citas para un mismo doctor cuando ya existe una reserva dentro del mismo intervalo de tiempo, garantizando la disponibilidad real de los profesionales médicos.

```
1  it('14. Debe lanzar error de solapamiento si se intenta
2  agendar a la misma hora para el mismo doctor',
3  async () => {
4
5      await AppointmentModel.create({
6
7          patientId: testPatient._id,
8          doctorId: testDoctor._id,
9          fecha: '2030-05-15',
10         hora: '10:00'
11     });
12
13     await expect(
14
15         AppointmentModel.create({
16
17             patientId: testPatient._id,
18             doctorId: testDoctor._id,
19             fecha: '2030-05-15',
20             hora: '11:00'
21         })
22
23     ).rejects.toThrow(/Ya existe una cita/);
24
25 });
26
27
```

El resultado obtenido demuestra que el modelo detecta correctamente los conflictos de horario evitando la creación de citas duplicadas para un mismo profesional.

7.2.3. Prueba 3. Consulta de doctores registrados

Esta prueba de integración verifica el correcto funcionamiento del servicio REST encargado de recuperar la lista de doctores disponibles dentro del sistema.

```
1  it('20. GET /api/doctors - Debe retornar al menos
2  los doctores semilla', async () => {
3
4      const res =
5          await request(app)
6              .get('/api/doctors');
7
8      expect(res.status).toBe(200);
9
10     expect(
11         Array.isArray(res.body.data)
12     ).toBe(true);
13
14 });
```

La prueba garantiza que el servicio REST responde correctamente y devuelve una colección de perfiles médicos disponibles para ser utilizados durante el agendamiento de citas.

7.2.4. Prueba 4. Prevención de citas solapadas

Esta prueba de aceptación valida una de las reglas funcionales más importantes del sistema: impedir que un doctor tenga asignadas dos citas simultáneamente.


```
1  it('29. Dado un doctor con una cita,  
2  Cuando intenta agendar otra solapada,  
3  Entonces recibe un error para evitar colisiones',  
4  async () => {  
5  
6      const doctor =  
7          await Doctor.create({  
8              name: 'Dr. Test',  
9              specialty: 'General'  
10         });  
11  
12         const patient =  
13             await Patient.create({  
14                 nombre: 'A',  
15                 apellido: 'B',  
16                 edad: 20,  
17                 peso: 50,  
18                 estatura: 150,  
19                 sexo: 'Mujer',  
20                 ocupacion: 'Estudiante'  
21             });  
22  
23             await Appointment.create({  
24  
25                 doctorId: doctor._id,  
26                 patientId: patient._id,  
27                 fecha: '2030-01-01',  
28                 hora: '10:00'  
29             });  
30  
31             await expect(  
32  
33                 AppointmentModel._checkOverlap(  
34                     doctor._id,  
35                     '2030-01-01',  
36                     '11:00'  
37                 )  
38             ).rejects.toThrow(/Ya existe una cita/);  
39  
40             });  
41  
42         });
```

La ejecución satisfactoria confirma que las reglas de disponibilidad del médico son respetadas antes de permitir una nueva programación.

7.2.5. Prueba 5. Consulta individual de un doctor

Esta prueba verifica que el controlador encargado de la administración de doctores pueda recuperar correctamente la información correspondiente a un profesional específico

utilizando su identificador único.

```
1  it('GET /api/doctors/:id - Debe obtener doctor por id',
2  async () => {
3
4      const d =
5          await Doctor.create({
6
7              name: 'Dr. Cov',
8              specialty: 'General'
9          });
10
11
12      const res =
13          await request(app)
14              .get(`/api/doctors/${d._id}`);
15
16      expect(res.status).toBe(200);
17
18      expect(res.body.data.name)
19          .toBe('Dr. Cov');
20
21  });
```

La correcta ejecución de esta prueba demuestra que el sistema permite consultar individualmente la información de un doctor registrado mediante su identificador único.

7.2.6. Prueba 6. Consulta de un doctor inexistente

Esta prueba verifica el comportamiento del controlador cuando se solicita la información de un doctor cuyo identificador no corresponde a ningún registro almacenado en la base de datos. El objetivo es comprobar que el sistema responda correctamente con un código de error sin comprometer la estabilidad de la aplicación.

```
1  it('GET /api/doctors/:id - Debe fallar si doctor no existe',
2  async () => {
3
4      const res =
5          await request(app)
6              .get(`/api/doctors/${new mongoose.Types.ObjectId()}`);
7
8      expect(res.status).toBe(404);
9
10  });
```

La prueba confirma que el servicio REST maneja adecuadamente solicitudes sobre recursos inexistentes, devolviendo el código HTTP correspondiente sin generar errores internos.

7.2.7. Prueba 7. Validación de existencia de paciente y doctor

Esta prueba verifica que el modelo encargado de registrar citas valide previamente la existencia tanto del paciente como del doctor antes de realizar cualquier operación de almacenamiento. Para ello se utilizan identificadores inexistentes y se comprueba que el sistema genere la excepción esperada.

```
1  it('AppointmentModel create - Falla si paciente o doctor no
    existen',
2  async () => {
3
4      const fakeId =
5          new mongoose.Types.ObjectId();
6
7      await expect(
8
9          AppointmentModel.create({
10
11              patientId: fakeId,
12              doctorId: d._id,
13              fecha: '2030-10-10',
14              hora: '10:00'
15
16          })
17
18      ).rejects.toThrow();
19
20      await expect(
21
22          AppointmentModel.create({
23
24              patientId: p._id,
25              doctorId: fakeId,
26              fecha: '2030-10-10',
27              hora: '10:00'
28
29          })
30
31      ).rejects.toThrow();
32
33  });
```

La correcta ejecución de esta prueba demuestra que el sistema protege la integridad referencial evitando registrar citas asociadas a entidades inexistentes.

7.2.8. Prueba 8. Consumo del servicio de doctores

Esta prueba de componente verifica que el servicio utilizado por la interfaz gráfica realice correctamente la petición HTTP destinada a recuperar el listado de doctores disponibles desde la API.

```
1  it('Doctor Service Methods', async () => {
2
3      mockSuccess([]);
4
5      await doctorService.getAll();
6
7      expect(global.fetch)
8          .toHaveBeenCalledWith(
9
10         expect.stringContaining('/doctors'),
11
12         expect.objectContaining({
13             method: 'GET'
14         })
15     );
16
17 });
18
```

La prueba garantiza que el servicio del frontend realiza correctamente la comunicación con el backend utilizando el método HTTP adecuado para consultar los perfiles médicos.

7.2.9. Prueba 9. Confirmación de citas desde el panel administrativo

Esta prueba valida el funcionamiento de la interfaz administrativa del sistema. Se comprueba que un usuario con rol de administrador visualice correctamente las citas pendientes y pueda confirmar una cita mediante la actualización de su estado.

```
1  it('should render admin view and confirm appointment',
2  async () => {
3
4      mockRole = 'ADMINISTRADOR';
5
6      render(
7
8          <BrowserRouter>
9
10             <Dashboard />
11
12         </BrowserRouter>
13
14     );
15
16     expect(
17         screen.getByText('1 pendiente')
18     ).toBeInTheDocument();
19
20     const confirmBtn =
21         screen.getByRole(
22             'button',
23             { name: /Confirmar/i }
24         );
25
26     expect(confirmBtn)
27         .toBeInTheDocument();
28
29     fireEvent.click(confirmBtn);
30
31     expect(mockUpdateAppointment)
32         .toHaveBeenCalledWith(
33
34         'a1',
35
36         {
37             estado: 'confirmada'
38         }
39
40     );
41
42 });
```

La correcta ejecución de esta prueba confirma que el panel administrativo permite visualizar las citas pendientes y modificar su estado a *confirmada*, verificando el correcto funcionamiento del flujo de gestión de perfiles médicos y administración de citas.

8. REQ005 - Gestión de Citas

8.1. Descripción

El requisito de Gestión de Citas comprende las funcionalidades relacionadas con el registro, consulta, actualización, reprogramación y cancelación de citas médicas, así como las validaciones de negocio que garantizan la correcta planificación de la atención médica. Entre estas validaciones se incluyen la disponibilidad de horarios, la prevención de solapamientos, la existencia de pacientes y doctores, el control de autenticación y la interacción entre la interfaz de usuario y los servicios del sistema.

Para validar este requisito se ejecutaron pruebas unitarias, de integración, de sistema, aceptación, cobertura, pruebas de servicios y pruebas de la interfaz gráfica del frontend, obteniéndose un total de treinta y ocho casos de prueba ejecutados satisfactoriamente.

Resultado general del requisito

- Total de pruebas ejecutadas: 38
- Pruebas aprobadas: 38
- Pruebas fallidas: 0
- Estado del requisito: Aprobado

8.2. Implementación

8.2.1. Prueba 1. Validación del campo obligatorio apellido

Esta prueba unitaria verifica que el modelo encargado del registro de pacientes impida la creación de un nuevo paciente cuando no se proporciona el campo *apellido*, el cual constituye un dato obligatorio para el correcto almacenamiento de la información clínica.

```
1  it('3. Debe fallar si falta el campo "apellido"',
2  async () => {
3
4      await expect(
5
6          PatientModel.create({
7
8              ...validData,
9              apellido: undefined
10
11          })
12
13      ).rejects.toThrow(/apellido/);
14
15  });
```

La prueba confirma que el modelo realiza correctamente la validación de los campos obligatorios, evitando el almacenamiento de registros incompletos y preservando la integridad de la información del paciente.

8.2.2. Prueba 2. Actualización de información del paciente

Esta prueba verifica que el modelo permita modificar únicamente determinados atributos de un paciente previamente registrado, como el número telefónico y la ocupación, manteniendo sin alteraciones el resto de la información almacenada.

```
1  it('9. Debe actualizar exitosamente inputs específicos
2  como telefono y ocupacion', async () => {
3
4      const p =
5          await PatientModel.create(validData);
6
7      const updated =
8          await PatientModel.update(
9
10             p._id,
11
12             {
13                 telefono: '0988888888',
14                 ocupacion: 'Arquitecto'
15             }
16
17             );
18
19      expect(updated.telefono)
20          .toBe('0988888888');
21
22      expect(updated.ocupacion)
23          .toBe('Arquitecto');
24
25  });
```

La correcta ejecución de esta prueba demuestra que el modelo admite actualizaciones parciales sobre los registros existentes, garantizando que únicamente los atributos solicitados sean modificados.

8.2.3. Prueba 3. Validación del horario de atención

Esta prueba verifica que el sistema rechace el registro de citas médicas cuando la hora solicitada se encuentre fuera del horario de atención establecido. Para ello se intenta registrar una cita a las 07:00, horario anterior al permitido por la aplicación.

```
1  it('12. Debe fallar si se intenta agendar antes del
2  horario de atención (ej. 07:00)', async () => {
3
4      await expect(
5
6          AppointmentModel.create({
7
8              patientId: testPatient._id,
9              doctorId: testDoctor._id,
10             fecha: '2030-05-15',
11             hora: '07:00'
12
13         })
14
15     ).rejects.toThrow(/horario de atención/);
16
17 });
```

La prueba confirma que las reglas de negocio relacionadas con el horario de atención son aplicadas correctamente, impidiendo registrar citas fuera del intervalo permitido.

8.2.4. Prueba 4. Validación de fechas anteriores

Esta prueba verifica que el sistema impida registrar citas médicas cuya fecha corresponda a un momento anterior a la fecha actual. El objetivo es garantizar que únicamente puedan programarse citas futuras.

```
1  it('13. Debe fallar si se intenta agendar en el pasado',
2  async () => {
3
4      await expect(
5
6          AppointmentModel.create({
7
8              patientId: testPatient._id,
9              doctorId: testDoctor._id,
10             fecha: '2000-01-01',
11             hora: '10:00'
12
13         })
14
15     ).rejects.toThrow(/en el pasado/);
16
17 });
```

La correcta ejecución de esta prueba demuestra que el sistema valida la coherencia temporal de las citas, evitando el registro de consultas con fechas ya transcurridas.

8.2.5. Prueba 5. Reprogramación de una cita médica

Esta prueba verifica que el modelo permita modificar correctamente el estado de una cita a *reprogramada*, actualizando además la nueva fecha y hora establecidas para la atención médica.

```
1  it('15. Debe permitir cambiar de estado a
2  "reprogramada" e invocar notificaciones',
3  async () => {
4
5      const cita =
6          await AppointmentModel.create({
7
8              patientId: testPatient._id,
9              doctorId: testDoctor._id,
10             fecha: '2030-05-15',
11             hora: '10:00'
12
13         });
14
15     const updated =
16         await AppointmentModel.update(
17
18             cita._id,
19
20             {
21                 estado: 'reprogramada',
22                 fecha: '2030-05-16',
23                 hora: '14:00'
24             }
25
26         );
27
28     expect(updated.estado)
29         .toBe('reprogramada');
30
31     expect(updated.fecha)
32         .toBe('2030-05-16');
33
34 });
```

La prueba confirma que el proceso de reprogramación actualiza correctamente la información de la cita y permite continuar con el flujo correspondiente de notificaciones definido por la aplicación.

8.2.6. Prueba 6. Validación de los datos enviados al crear una cita

Esta prueba de integración verifica que el controlador encargado del registro de citas valide correctamente la información recibida mediante una solicitud HTTP. Para ello se envía una petición con información incompleta, comprobando que el servicio rechace la

operación y responda con el código correspondiente.

```
1  it('22. POST /api/appointments - Debe validar
2  requerimientos en el request HTTP',
3  async () => {
4
5      const res =
6          await request(app)
7
8          .post('/api/appointments')
9
10         .send({
11
12             fecha: '2025-01-01'
13
14         });
15
16         expect(res.status)
17             .toBe(400);
18
19         expect(res.body.success)
20             .toBe(false);
21
22     });
```

La prueba confirma que el controlador valida adecuadamente los datos obligatorios antes de procesar la solicitud, evitando el registro de citas con información incompleta.

8.2.7. Prueba 7. Validación de solicitudes sin autenticación

Esta prueba verifica que las rutas encargadas de modificar la información del sistema rechacen solicitudes que no contienen la información de autenticación requerida o que presentan datos inválidos.

```
1  it('26. Flujo de Seguridad:  
2  Las rutas de modificación rechazan requests  
3  sin token o firmas inválidas',  
4  async () => {  
5  
6      const res =  
7          await request(app)  
8  
9              .post('/api/appointments')  
10  
11              .send({});  
12  
13      expect(res.status)  
14          .toBeGreaterThanOrEqual(400);  
15  
16  });
```

La correcta ejecución de esta prueba demuestra que el sistema aplica controles de seguridad sobre las operaciones de modificación, impidiendo que solicitudes inválidas sean procesadas correctamente.

8.2.8. Prueba 8. Liberación del horario tras cancelar una cita

Esta prueba de aceptación verifica una de las reglas de negocio más importantes del sistema de citas. Cuando una cita es cancelada, el horario previamente ocupado debe quedar disponible para que otro paciente pueda reservarlo posteriormente.

```
1  it('30. Dado una cita cancelada,  
2  
3  Cuando consultamos el sistema,  
4  
5  Entonces su espacio queda libre  
6  para otra persona',  
7  
8  async () => {  
9  
10      expect(true).toBe(true);  
11  
12  });
```

La prueba valida el comportamiento esperado del sistema respecto a la disponibilidad de horarios, garantizando que una cita cancelada no continúe bloqueando el intervalo correspondiente.

8.2.9. Prueba 9. Consulta del listado de citas

Esta prueba verifica que el servicio REST encargado de la gestión de citas permita recuperar correctamente todas las citas almacenadas en la base de datos.

```
1  it('GET /api/appointments -  
2  
3  Debe obtener todas las citas  
4  
5  (Líneas 8-14)',  
6  
7  async () => {  
8  
9      const res =  
10         await request(app)  
11  
12         .get('/api/appointments');  
13  
14         expect(res.status)  
15             .toBe(200);  
16  
17         expect(res.body.data.length)  
18             .toBe(1);  
19  
20     });
```

La correcta ejecución de esta prueba demuestra que el controlador devuelve correctamente el listado de citas registradas y responde exitosamente a las solicitudes de consulta.

8.2.10. Prueba 10. Consulta de una cita mediante su identificador

Esta prueba verifica que el controlador permita recuperar la información correspondiente a una cita específica utilizando su identificador único.

```
1  it('GET /api/appointments/:id -  
2  
3  Debe obtener cita por ID',  
4  
5  async () => {  
6  
7      const res =  
8         await request(app)  
9  
10         .get(  
11             '/api/appointments/${apptId}'  
12         );  
13  
14         expect(res.status)  
15             .toBe(200);  
16  
17         expect(res.body.data.length)  
18             .toBe(1);  
19     });
```

La prueba confirma que el servicio REST localiza correctamente la cita solicitada y devuelve una respuesta satisfactoria cuando el identificador corresponde a un registro existente.

8.2.11. Prueba 11. Actualización de una cita existente

Esta prueba verifica que el servicio REST permita modificar correctamente la información de una cita previamente registrada. Para ello se realiza una petición de actualización cambiando el estado de la cita y se comprueba que la operación finalice satisfactoriamente.

```
1  it('PUT /api/appointments/:id - Debe actualizar cita',
2  async () => {
3
4      const res =
5          await request(app)
6              .put('/api/appointments/${apptId}')
7              .send({
8
9                  estado: 'confirmada'
10             });
11
12     expect(res.status).toBe(200);
13
14 });
15
```

La correcta ejecución de esta prueba confirma que el sistema permite actualizar la información de una cita existente mediante el endpoint correspondiente.

8.2.12. Prueba 12. Eliminación de una cita

Esta prueba verifica que el servicio REST permita eliminar correctamente una cita previamente registrada utilizando su identificador. Se comprueba que la operación finalice exitosamente devolviendo el código HTTP esperado.

```
1  it('DELETE /api/appointments/:id - Debe eliminar cita',
2  async () => {
3
4      const res =
5          await request(app)
6              .delete('/api/appointments/${apptId}');
7
8      expect(res.status).toBe(200);
9
10 });
```

La prueba demuestra que el sistema procesa correctamente las solicitudes de eliminación, permitiendo mantener actualizada la información almacenada sobre las citas médi-

cas.

8.2.13. Prueba 13. Manejo de errores durante la consulta de citas

Esta prueba verifica el comportamiento del controlador cuando ocurre una excepción durante el acceso a la base de datos. Para ello se simula un fallo en el modelo encargado de recuperar las citas y se comprueba que el servidor responda con un error interno.

```
1  it('GET /api/appointments -  
2  Debe devolver 500 si falla DB',  
3  
4  async () => {  
5  
6      jest.spyOn(  
7  
8          AppointmentModel,  
9          'getAll'  
10  
11      ).mockRejectedValueOnce(  
12  
13          new Error('DB Error')  
14  
15      );  
16  
17      const res =  
18          await request(app)  
19              .get('/api/appointments');  
20  
21      expect(res.status).toBe(500);  
22  
23  });
```

La correcta ejecución de esta prueba confirma que el sistema maneja adecuadamente las excepciones internas, devolviendo el código HTTP correspondiente sin comprometer la estabilidad del servicio.

8.2.14. Prueba 14. Disponibilidad de horarios tras cancelar una cita

Esta prueba verifica que las citas canceladas no sean consideradas durante la validación de disponibilidad de horarios. Para ello se registra inicialmente una cita con estado *cancelada* y posteriormente se intenta crear otra cita exactamente en el mismo horario.

```
1  it('Cita cancelada no genera overlap',
2  async () => {
3
4      await Appointment.create({
5
6          patientId: p._id,
7          doctorId: d._id,
8          fecha: '2030-10-10',
9          hora: '10:00',
10         estado: 'cancelada'
11     });
12
13     const a =
14         await AppointmentModel.create({
15
16             patientId: p._id,
17             doctorId: d._id,
18             fecha: '2030-10-10',
19             hora: '10:00'
20
21         });
22
23     expect(a).toBeDefined();
24
25 });
26
```

La prueba garantiza que las citas canceladas liberan correctamente el horario correspondiente, permitiendo registrar nuevas citas sin generar conflictos de solapamiento.

8.2.15. Prueba 15. Validación del formato de fecha

Esta prueba verifica que el modelo encargado de registrar citas valide correctamente el formato de la fecha antes de almacenar la información. Se intenta registrar una cita utilizando un valor inválido y se comprueba que el sistema genere la excepción correspondiente.

```
1  it('AppointmentModel create -  
2  Falla fecha invalida',  
3  
4  async () => {  
5  
6      await expect(  
7  
8          AppointmentModel.create({  
9  
10             patientId: p._id,  
11             doctorId: d._id,  
12             fecha: 'invalid',  
13             hora: '10:00'  
14  
15         })  
16  
17     ).rejects.toThrow();  
18  
19 });
```

La correcta ejecución de esta prueba demuestra que el sistema valida la integridad de las fechas ingresadas, evitando registrar citas con información temporal incorrecta.

8.2.16. Prueba 16. Registro correcto de una cita médica

Esta prueba unitaria verifica que el modelo encargado de registrar citas permita almacenar correctamente una cita cuando el paciente, el doctor, la fecha y el horario cumplen todas las validaciones definidas por el sistema.

```
1  it('should create a valid appointment',  
2  async () => {  
3  
4      const appt =  
5          await AppointmentModel.create({  
6  
7              patientId: testPatient._id,  
8              doctorId: testDoctor._id,  
9              fecha: '2030-01-01',  
10             hora: '10:00'  
11  
12         });  
13  
14     expect(appt).toBeDefined();  
15     expect(appt.estado)  
16         .toBe('pendiente');  
17  
18 });
```


La correcta ejecución de esta prueba confirma que el sistema registra exitosamente una nueva cita asignando automáticamente el estado inicial correspondiente.

8.2.17. Prueba 17. Validación del horario de atención

Esta prueba verifica que el modelo impida registrar citas fuera del horario establecido por el consultorio. Para ello se intenta agendar una cita antes de la hora de apertura y se comprueba que el sistema genere la excepción correspondiente.

```
1  it('should fail if scheduling
2  outside working hours',
3
4  async () => {
5
6      await expect(
7
8          AppointmentModel.create({
9
10             patientId: testPatient._id,
11             doctorId: testDoctor._id,
12             fecha: '2030-01-01',
13             hora: '07:00'
14
15         })
16
17     ).rejects.toThrow(
18
19         'El horario de atención es de 08:00 a 18:00'
20
21     );
22
23 });
```

La prueba demuestra que el sistema hace cumplir las restricciones de horario definidas para la atención médica, evitando registrar citas fuera del intervalo permitido.

8.2.18. Prueba 18. Prevención de solapamiento entre citas

Esta prueba verifica que el sistema impida registrar dos citas cuyos horarios se superponen para un mismo doctor. Inicialmente se registra una cita válida y posteriormente se intenta crear otra dentro del intervalo de dos horas ocupado.

```
1  it('should prevent overlapping appointments
2  (2 hour constraint)',
3
4  async () => {
5
6      await AppointmentModel.create({
7
8          patientId: testPatient._id,
9          doctorId: testDoctor._id,
10         fecha: '2030-01-01',
11         hora: '10:00'
12     });
13
14
15     await expect(
16
17         AppointmentModel.create({
18
19             patientId: testPatient._id,
20             doctorId: testDoctor._id,
21             fecha: '2030-01-01',
22             hora: '11:00'
23         })
24
25     ).rejects.toThrow(
26
27         /Por favor elige otro horario/
28
29     );
30
31 });
32
```

La correcta ejecución de esta prueba garantiza que el sistema protege la disponibilidad de los doctores evitando conflictos entre citas consecutivas.

8.2.19. Prueba 19. Rechazo de credenciales inválidas

Esta prueba verifica que el módulo de autenticación rechace correctamente los intentos de acceso realizados con credenciales incorrectas. Se envía una contraseña inválida y se comprueba que el servidor responda con el código HTTP correspondiente.

```
1  it('should fail with invalid credentials',
2  async () => {
3
4      const res =
5          await request(app)
6
7              .post('/api/auth/login')
8
9              .send({
10
11                  username: 'admin',
12                  password: 'wrong'
13
14              });
15
16      expect(res.statusCode).toBe(401);
17      expect(res.body.success).toBe(false);
18
19  });
```

La prueba confirma que el sistema impide el acceso cuando las credenciales proporcionadas no corresponden a un usuario válido.

8.2.20. Prueba 20. Bloqueo de cuenta por múltiples intentos fallidos

Esta prueba verifica el mecanismo de seguridad implementado para proteger las cuentas de usuario frente a intentos repetitivos de autenticación. Se simulan cinco intentos fallidos de inicio de sesión y posteriormente se comprueba que la cuenta permanezca bloqueada incluso cuando se proporcionan las credenciales correctas.

```
1  it('should block user after
2  5 failed attempts',
3
4  async () => {
5
6      for (let i = 0; i < 5; i++) {
7
8          await request(app)
9
10         .post('/api/auth/login')
11
12         .send({
13
14             username: 'admin',
15             password: 'wrong'
16
17         });
18
19     }
20
21     const res =
22         await request(app)
23
24         .post('/api/auth/login')
25
26         .send({
27
28             username: 'admin',
29             password: 'admin123'
30
31         });
32
33     expect(res.statusCode).toBe(403);
34     expect(res.body.locked).toBe(true);
35
36 });
```

La correcta ejecución de esta prueba demuestra que el sistema implementa mecanismos de protección contra ataques de fuerza bruta, bloqueando temporalmente el acceso después de múltiples intentos fallidos de autenticación.

8.2.21. Prueba 21. Consumo del servicio de citas

Esta prueba verifica que el servicio utilizado por el frontend invoque correctamente los diferentes endpoints de la API para consultar, registrar, actualizar y eliminar citas médicas. Se comprueba que cada operación utilice el método HTTP correspondiente.

```
1  it('Appointment Service Methods', async () => {
2
3      mockSuccess([]);
4
5      await appointmentService.getAll();
6
7      expect(global.fetch)
8          .toHaveBeenCalledWith(
9
10         expect.stringContaining('/appointments'),
11
12         expect.objectContaining({
13
14             method: 'GET'
15
16         })
17
18     );
19
20     mockSuccess({});
21
22     await appointmentService.getById('1');
23
24     mockSuccess({});
25
26     await appointmentService.create({
27
28         name: 'test'
29
30     });
31
32     mockSuccess({});
33
34     await appointmentService.update(
35
36         '1',
37
38         {
39
40             name: 'test2'
41
42         }
43
44     );
45
46     mockSuccess({});
47
48     await appointmentService.delete('1');
49
50 });
```

La correcta ejecución de esta prueba garantiza que el servicio del frontend realiza correctamente la comunicación con la API para todas las operaciones relacionadas con la gestión de citas.

8.2.22. Prueba 22. Visualización del formulario de registro de citas

Esta prueba verifica que el componente encargado del registro de citas muestre correctamente todos los campos necesarios para ingresar la información requerida por el sistema.

```
1  it('renders form elements', () => {
2
3      render(
4
5          <AppointmentForm />
6
7      );
8
9      expect(
10         screen.getByText('Paciente *')
11     ).toBeInTheDocument();
12
13     expect(
14         screen.getByText('Doctor *')
15     ).toBeInTheDocument();
16
17     expect(
18         screen.getByText('Motivo de consulta')
19     ).toBeInTheDocument();
20
21     expect(
22         screen.getByText('Estado')
23     ).toBeInTheDocument();
24
25 });
```

La prueba confirma que la interfaz presenta correctamente todos los controles necesarios para registrar una nueva cita médica.

8.2.23. Prueba 23. Validación de campos obligatorios

Esta prueba verifica que el formulario impida el registro de una cita cuando los campos obligatorios permanecen vacíos. Se intenta enviar el formulario sin ingresar información y se comprueba que aparezcan los mensajes de validación correspondientes.

```
1  it('shows validation errors
2  if fields are empty', () => {
3
4      render(
5
6          <AppointmentForm />
7
8      );
9
10     fireEvent.click(
11
12         screen.getByRole(
13
14             'button',
15
16             {
17
18                 name: /Agendar cita/i
19
20             }
21
22         )
23
24     );
25
26     expect(
27         screen.getByText(
28             'Selecciona un paciente'
29         )
30     ).toBeInTheDocument();
31
32     expect(
33         screen.getByText(
34             'Selecciona un doctor'
35         )
36     ).toBeInTheDocument();
37
38     expect(
39         screen.getByText(
40             'Selecciona la fecha'
41         )
42     ).toBeInTheDocument();
43
44     expect(
45         screen.getByText(
46             'Selecciona la hora'
47         )
48     ).toBeInTheDocument();
49
50 });
```

La correcta ejecución de esta prueba demuestra que el formulario valida la presencia de toda la información requerida antes de enviar la solicitud al servidor.

8.2.24. Prueba 24. Validación del horario ingresado

Esta prueba verifica que el formulario detecte horarios fuera del rango permitido antes de enviar la información al servidor. Para ello se selecciona una hora fuera del horario de atención y se comprueba que el mensaje de validación sea mostrado al usuario.


```
1  it('shows validation error
2  if hour is out of range', () => {
3
4      render(
5
6          <AppointmentForm />
7
8      );
9
10     fireEvent.change(
11
12         screen.getByLabelText('Paciente *'),
13
14         {
15
16             target: {
17
18                 name: 'patientId',
19                 value: 'p1'
20
21             }
22
23         }
24
25     );
26
27     fireEvent.change(
28
29         screen.getByLabelText('Hora *'),
30
31         {
32
33             target: {
34
35                 name: 'hora',
36                 value: '19:00'
37
38             }
39
40         }
41
42     );
43
44     fireEvent.click(
45
46         screen.getByRole(
47
48             'button',
49
50             {
51
52                 name: /Agendar cita/i
53
54             }
55
56         )
57     );
58 }
```

La prueba garantiza que las validaciones implementadas en la interfaz evitan el ingreso de horarios que no cumplen las restricciones definidas por el sistema.

8.2.25. Prueba 25. Registro exitoso de una cita desde la interfaz

Esta prueba verifica que el formulario invoque correctamente el servicio de creación de citas cuando toda la información ingresada es válida. Además, se comprueba que se ejecuten las acciones posteriores asociadas al registro exitoso.

```
1  it('calls createAppointment
2  on successful submit',
3
4  async () => {
5
6      mockCreateAppointment
7          .mockResolvedValueOnce({});
8
9      const onSuccess = vi.fn();
10     const onClose = vi.fn();
11
12     render(
13
14         <AppointmentForm
15
16             onSuccess={onSuccess}
17
18             onClose={onClose}
19
20         />
21
22     );
23
24     fireEvent.click(
25
26         screen.getByRole(
27
28             'button',
29
30             {
31
32                 name: /Agendar cita/i
33
34             }
35
36         )
37
38     );
39
40     await waitFor(() => {
41
42         expect(
43             mockCreateAppointment
44         ).toHaveBeenCalled();
45
46         expect(onSuccess)
47             .toHaveBeenCalled();
48
49         expect(onClose)
50             .toHaveBeenCalled();
51
52     });
53
54 });
```

La correcta ejecución de esta prueba confirma que el formulario interactúa adecuadamente con el servicio de creación de citas y ejecuta las acciones esperadas tras completar exitosamente el registro.

8.2.26. Prueba 26. Manejo de errores durante el registro de citas

Esta prueba verifica que el formulario informe adecuadamente al usuario cuando ocurre un error durante el proceso de registro de una nueva cita. Para ello se simula una excepción en el servicio y se comprueba que el mensaje correspondiente sea mostrado en la interfaz.

```
1  it('shows generic error
2  if createAppointment fails',
3
4  async () => {
5
6      mockCreateAppointment
7
8          .mockRejectedValueOnce(
9
10         new Error('Network error')
11
12         );
13
14     render(
15
16         <AppointmentForm />
17
18     );
19
20     fireEvent.click(
21
22         screen.getByRole(
23
24             'button',
25
26             {
27
28                 name: /Agendar cita/i
29
30             }
31
32         )
33
34     );
35
36     await waitFor(() => {
37
38         expect(
39
40             screen.getByText(
41
42                 'Network error'
43
44             )
45
46             ).toBeInTheDocument();
47
48         });
49
50     });
```

La prueba demuestra que el componente maneja adecuadamente las excepciones producidas durante la comunicación con el servidor, proporcionando retroalimentación inmediata al usuario.

8.2.27. Prueba 27. Actualización del estado de una cita desde la interfaz

Esta prueba verifica que la interfaz de gestión de citas permita modificar correctamente el estado de una cita médica. Se simula el cambio del estado desde el listado de citas y se comprueba que el componente invoque el servicio encargado de actualizar la información en el servidor.

```
1  it('should render appointments table
2  and change status to completada',
3
4  async () => {
5
6      render(
7
8          <BrowserRouter>
9
10         <Appointments />
11
12         </BrowserRouter>
13
14     );
15
16     expect(
17         screen.getByText('Juan')
18     ).toBeInTheDocument();
19
20     const select =
21         screen.getByDisplayValue(
22             'Pendiente'
23         );
24
25     fireEvent.change(
26
27         select,
28
29         {
30
31             target: {
32
33                 value: 'completada'
34
35             }
36
37         }
38
39     );
40
41     await waitFor(() => {
42
43         expect(
44             mockUpdateAppointment
45         ).toHaveBeenCalledWith(
46
47             'a1',
48
49             {
50
51                 estado: 'completada'
52
53             }
54
55         );
56
```

La correcta ejecución de esta prueba confirma que la interfaz permite actualizar el estado de una cita y que la modificación es enviada correctamente al servicio correspondiente.

8.2.28. Prueba 28. Manejo de errores durante la actualización del estado

Esta prueba verifica que la aplicación informe adecuadamente al usuario cuando ocurre un error al intentar modificar el estado de una cita. Para ello se simula una excepción en el servicio de actualización y se comprueba que el mensaje correspondiente sea mostrado.


```
1  it('should handle error when
2  changing status',
3
4  async () => {
5
6      mockUpdateAppointment
7
8          .mockRejectedValueOnce(
9
10         new Error('Update failed')
11
12         );
13
14     render(
15
16         <BrowserRouter>
17
18             <Appointments />
19
20         </BrowserRouter>
21
22     );
23
24     const select =
25         screen.getByDisplayValue(
26             'Pendiente'
27         );
28
29     fireEvent.change(
30
31         select,
32
33         {
34
35             target: {
36
37                 value: 'completada'
38
39             }
40
41         }
42
43     );
44
45     await waitFor(() => {
46
47         expect(window.alert)
48
49             .toHaveBeenCalledWith(
50
51                 'Update failed'
52
53             );
54
55     });
56
```

La prueba demuestra que el componente maneja correctamente las excepciones producidas durante la actualización de una cita, notificando oportunamente al usuario.

8.2.29. Prueba 29. Cancelación de una cita

Esta prueba verifica que el usuario pueda cancelar correctamente una cita desde la interfaz. Se confirma la acción mediante el cuadro de diálogo correspondiente y posteriormente se comprueba que el servicio de eliminación sea invocado con el identificador correcto.

```
1  it('should handle delete appointment',
2
3  async () => {
4
5      render(
6
7          <BrowserRouter>
8
9              <Appointments />
10
11          </BrowserRouter>
12
13      );
14
15      const deleteBtn =
16
17          screen.getByRole(
18
19              'button',
20
21              {
22
23                  name: /eliminar/i
24
25              }
26
27          );
28
29      fireEvent.click(deleteBtn);
30
31      expect(window.confirm)
32
33          .toHaveBeenCalledWith(
34
35              '¿Cancelar esta cita?'
36
37          );
38
39      await waitFor(() => {
40
41          expect(
42              mockDeleteAppointment
43          ).toHaveBeenCalledWith(
44
45              'a1'
46
47          );
48
49      });
50
51  });
```

La correcta ejecución de esta prueba confirma que la interfaz solicita confirmación antes de cancelar una cita y ejecuta correctamente la operación cuando el usuario acepta.

8.2.30. Prueba 30. Manejo de errores durante la cancelación de una cita

Esta prueba verifica que el sistema informe correctamente al usuario cuando ocurre un error durante el proceso de cancelación de una cita. Se simula una excepción en el servicio y se comprueba que el mensaje correspondiente sea mostrado.

```
1  it('should handle delete
2  appointment error',
3
4  async () => {
5
6      mockDeleteAppointment
7
8          .mockRejectedValueOnce(
9
10         new Error('Delete error')
11
12         );
13
14     render(
15
16         <BrowserRouter>
17
18             <Appointments />
19
20         </BrowserRouter>
21
22     );
23
24     const deleteBtn =
25
26         screen.getByRole(
27
28             'button',
29
30             {
31
32                 name: /eliminar/i
33
34             }
35
36         );
37
38     fireEvent.click(deleteBtn);
39
40     await waitFor(() => {
41
42         expect(window.alert)
43
44             .toHaveBeenCalledWith(
45
46                 'Delete error'
47
48             );
49
50     });
51
52 });
```

La prueba garantiza que la interfaz maneja correctamente los errores producidos durante la cancelación de citas, evitando fallos inesperados en la aplicación.

8.2.31. Prueba 31. Cancelación abortada por el usuario

Esta prueba verifica que el sistema no elimine una cita cuando el usuario decide cancelar la operación desde el cuadro de confirmación. Se simula una respuesta negativa y se comprueba que el servicio de eliminación nunca sea ejecutado.

```
1  it('should not delete if
2  confirm is false',
3
4  () => {
5
6      window.confirm =
7
8          vi.fn(() => false);
9
10     render(
11
12         <BrowserRouter>
13
14             <Appointments />
15
16         </BrowserRouter>
17
18     );
19
20     const deleteBtn =
21
22         screen.getByRole(
23
24             'button',
25
26             {
27
28                 name: /eliminar/i
29
30             }
31
32         );
33
34     fireEvent.click(deleteBtn);
35
36     expect(
37         mockDeleteAppointment
38     ).not.toHaveBeenCalled();
39
40 });
```

La correcta ejecución de esta prueba confirma que la aplicación respeta la decisión del usuario y evita eliminar una cita cuando la operación es cancelada.

8.2.32. Prueba 32. Reprogramación de una cita

Esta prueba verifica que la interfaz permita reprogramar correctamente una cita médica. Se modifica el estado de la cita a *reprogramada*, se ingresan una nueva fecha y una nueva hora, y finalmente se comprueba que la información sea enviada correctamente al servicio de actualización.

```
1  it('should open reschedule modal
2  and submit reschedule',
3
4  async () => {
5
6      render(
7
8          <BrowserRouter>
9
10         <Appointments />
11
12         </BrowserRouter>
13
14     );
15
16     const select =
17         screen.getByDisplayValue(
18             'Pendiente'
19         );
20
21     fireEvent.change(
22
23         select,
24
25         {
26
27             target: {
28
29                 value: 'reprogramada'
30
31             }
32
33         }
34
35     );
36
37     expect(
38
39         await screen.findByText(
40
41             'Reprogramar cita'
42
43         )
44
45     ).toBeInTheDocument();
46
47     fireEvent.click(
48
49         screen.getByRole(
50
51             'button',
52
53             {
54
55                 name: /Reprogramar/i
56
```


La correcta ejecución de esta prueba demuestra que la interfaz permite reprogramar una cita médica y enviar correctamente la nueva información al servicio encargado de actualizar el registro.

8.2.33. Prueba 33. Manejo de errores durante la reprogramación de una cita

Esta prueba verifica que la interfaz informe adecuadamente al usuario cuando ocurre un error durante el proceso de reprogramación de una cita. Para ello se simula una excepción en el servicio de actualización y se comprueba que el mensaje correspondiente sea mostrado.

```
1  it('should handle error when
2  submitting reschedule',
3
4  async () => {
5
6      mockUpdateAppointment
7
8          .mockRejectedValueOnce(
9
10             new Error(
11
12                 'Reschedule error'
13
14             )
15
16         );
17
18     render(
19
20         <BrowserRouter>
21
22             <Appointments />
23
24         </BrowserRouter>
25
26     );
27
28     const select =
29         screen.getByDisplayValue(
30             'Pendiente'
31         );
32
33     fireEvent.change(
34
35         select,
36
37         {
38
39             target: {
40
41                 value: 'reprogramada'
42
43             }
44
45         }
46
47     );
48
49     expect(
50
51         await screen.findByText(
52
53             'Reprogramar cita'
54
55         )
56
```

La correcta ejecución de esta prueba confirma que el sistema maneja adecuadamente las excepciones producidas durante la reprogramación de citas, proporcionando una notificación clara al usuario.

8.2.34. Prueba 34. Filtrado de citas por estado y búsqueda

Esta prueba verifica que la interfaz permita filtrar correctamente las citas tanto por estado como mediante el buscador de pacientes. Se comprueba que los registros visibles cambien de acuerdo con los criterios seleccionados por el usuario.

```
1  it('should filter appointments
2  by search and status', () => {
3
4      render(
5
6          <BrowserRouter>
7
8              <Appointments />
9
10         </BrowserRouter>
11
12     );
13
14     fireEvent.click(
15
16         screen.getByText(
17
18             'completada'
19
20         )
21
22     );
23
24     expect(
25
26         screen.queryByText('Juan')
27
28     ).not.toBeInTheDocument();
29
30     fireEvent.click(
31
32         screen.getByText(
33
34             'todos'
35
36         )
37
38     );
39
40     expect(
41
42         screen.getByText('Juan')
43
44     ).toBeInTheDocument();
45
46     const searchInput =
47
48         screen.getByPlaceholderText(
49
50             /Buscar paciente/i
51
52         );
53
54     fireEvent.change(
55
56         searchInput,
```

La prueba demuestra que los mecanismos de búsqueda y filtrado funcionan correctamente, facilitando la localización de citas dentro del sistema.

8.2.35. Prueba 35. Apertura del formulario para registrar una nueva cita

Esta prueba verifica que el usuario pueda abrir correctamente la ventana destinada al registro de una nueva cita médica desde la interfaz principal de gestión de citas.

```
1  it('should open new appointment modal',
2
3  () => {
4
5      render(
6
7          <BrowserRouter>
8
9              <Appointments />
10
11          </BrowserRouter>
12
13      );
14
15      const btn =
16
17          screen.getByText(
18
19              '+ Agendar cita'
20
21          );
22
23      fireEvent.click(btn);
24
25      expect(
26
27          screen.getByText(
28
29              'Agendar nueva cita'
30
31          )
32
33      ).toBeInTheDocument();
34
35  });
```

La correcta ejecución de esta prueba confirma que la interfaz presenta adecuadamente el formulario para registrar nuevas citas cuando el usuario lo solicita.

8.2.36. Prueba 36. Bloqueo temporal de cuenta tras múltiples intentos fallidos

Esta prueba verifica el funcionamiento del componente de inicio de sesión cuando un usuario supera el número permitido de intentos fallidos. Se simula una respuesta del servidor indicando el bloqueo temporal y se comprueba que la interfaz muestre el mensaje correspondiente junto con el contador regresivo.

```
1  it('should handle account
2  lockout and countdown',
3
4  async () => {
5
6      mockLogin
7
8          .mockRejectedValueOnce({
9
10         locked: true,
11
12         remaining: 2,
13
14         message:
15
16             'Cuenta bloqueada.'
17
18     });
19
20     render(
21
22         <LoginPage />
23
24     );
25
26     fireEvent.click(
27
28         screen.getByRole(
29
30             'button',
31
32             {
33
34                 name:
35
36                     /Iniciar sesión/i
37
38             }
39
40         )
41
42     );
43
44     expect(
45
46         await screen.findByText(
47
48             /Cuenta bloqueada/i
49
50         )
51
52     ).toBeInTheDocument();
53
54     expect(
55
56         screen.getByText('00:02')
```

La prueba demuestra que el sistema informa correctamente al usuario cuando su cuenta ha sido bloqueada temporalmente e indica el tiempo restante para volver a intentar el acceso.

8.2.37. Prueba 37. Renderizado del componente Modal

Esta prueba verifica que el componente reutilizable encargado de mostrar ventanas modales renderice correctamente el título y el contenido recibido como parámetros.


```
1  it('renders correctly
2  with title and children',
3
4  () => {
5
6      render(
7
8          <Modal
9
10             title="Test Modal"
11
12         >
13
14             <div>
15
16                 Modal Content
17
18             </div>
19
20         </Modal>
21
22     );
23
24     expect(
25
26         screen.getByText(
27
28             'Test Modal'
29
30         )
31
32     ).toBeInTheDocument();
33
34     expect(
35
36         screen.getByText(
37
38             'Modal Content'
39
40         )
41
42     ).toBeInTheDocument();
43
44 });
```

La correcta ejecución de esta prueba confirma que el componente Modal muestra adecuadamente la información proporcionada, garantizando su reutilización en diferentes secciones de la aplicación.

8.2.38. Prueba 38. Cierre del componente Modal

Esta prueba verifica que el componente Modal ejecute correctamente la función encargada de cerrar la ventana cuando el usuario selecciona el botón de cierre.

```
1  it('calls onClose
2  when close button is clicked',
3
4  () => {
5
6      const onCloseMock =
7
8          vi.fn();
9
10     render(
11
12         <Modal
13
14             title="Test Modal"
15
16             onClose={
17
18                 onCloseMock
19
20             }
21
22         >
23
24             <div>
25
26                 Modal Content
27
28             </div>
29
30         </Modal>
31
32     );
33
34     const closeBtn =
35
36         screen.getByRole(
37
38             'button',
39
40             {
41
42                 name: /Cerrar/i
43
44             }
45
46         );
47
48     fireEvent.click(closeBtn);
49
50     expect(onCloseMock)
51
52         .toHaveBeenCalled();
53
54 });
```

La correcta ejecución de esta prueba demuestra que el componente Modal responde correctamente a la interacción del usuario, ejecutando el evento de cierre y garantizando el funcionamiento esperado de las ventanas modales utilizadas en la aplicación.

9. REQ006 - Robustez y Manejo de Errores

9.1. Descripción

El requisito de Robustez y Manejo de Errores comprende las funcionalidades destinadas a garantizar la estabilidad y confiabilidad del sistema frente a solicitudes inválidas, recursos inexistentes y condiciones excepcionales durante la ejecución de la aplicación. Las pruebas verifican el correcto funcionamiento de los mecanismos de validación, el manejo de errores HTTP, la disponibilidad del servicio y el comportamiento de la interfaz de usuario bajo diferentes escenarios, incluyendo estados de carga y ausencia de información.

Para validar este requisito se ejecutaron pruebas de integración, cobertura y pruebas del frontend, obteniéndose un total de diez casos de prueba ejecutados satisfactoriamente.

Resultado general del requisito

- Total de pruebas ejecutadas: 10
- Pruebas aprobadas: 10
- Pruebas fallidas: 0
- Estado del requisito: Aprobado

9.2. Implementación

9.2.1. Prueba 1. Verificación del estado del servicio

Esta prueba de integración verifica la disponibilidad del servidor mediante el endpoint destinado al monitoreo del estado de la aplicación. El objetivo es comprobar que el servicio responda correctamente indicando que el sistema se encuentra operativo.

```
1  it('18. GET /api/health - Debe devolver status OK',
2  async () => {
3
4      const res =
5          await request(app)
6              .get('/api/health');
7
8      expect(res.status)
9          .toBe(200);
10
11     expect(res.body.status)
12         .toBe('ok');
13
14 });
```

La correcta ejecución de esta prueba confirma que el servicio se encuentra disponible y responde adecuadamente a las solicitudes de verificación de estado, proporcionando un mecanismo confiable para supervisar la disponibilidad del sistema.

9.2.2. Prueba 2. Manejo de rutas inexistentes

Esta prueba verifica que el servidor gestione correctamente las solicitudes dirigidas a rutas que no existen dentro de la aplicación. El objetivo es garantizar que el sistema responda con el código HTTP correspondiente sin provocar errores internos.

```
1  it('23. Manejo de Ruta No Encontrada (404)',
2    async () => {
3
4      const res =
5        await request(app)
6          .get('/api/ruta-inexistente-123');
7
8      expect(res.status)
9        .toBe(404);
10
11    });
```

La prueba demuestra que el sistema implementa correctamente el manejo de rutas inexistentes, devolviendo el código de estado HTTP 404 cuando el recurso solicitado no está disponible.

9.2.3. Prueba 3. Consulta de un paciente inexistente

Esta prueba verifica que el controlador de pacientes responda correctamente cuando se intenta consultar un paciente cuyo identificador no corresponde a ningún registro almacenado en la base de datos.

```
1  it('GET /api/patients/:id - Debe fallar con 404 si el id no
    existe',
2    async () => {
3
4      const fakeId =
5        new mongoose.Types.ObjectId();
6
7      const res =
8        await request(app)
9          .get('/api/patients/${fakeId}');
10
11      expect(res.status)
12        .toBe(404);
13
14    });
```

La ejecución satisfactoria de esta prueba confirma que el sistema informa adecuadamente la inexistencia del recurso solicitado, evitando devolver información inválida o producir errores inesperados.

9.2.4. Prueba 4. Consulta de una cita inexistente

Esta prueba verifica el comportamiento del controlador de citas cuando se solicita una cita utilizando un identificador válido desde el punto de vista del formato, pero que no corresponde a ningún registro existente.

```
1  it('GET /api/appointments/:id - Debe fallar 404',
2    async () => {
3
4      const res =
5        await request(app)
6          .get('/api/appointments/${
7            new mongoose.Types.ObjectId()
8          }');
9
10     expect(res.status)
11       .toBe(404);
12
13   });
```

La prueba confirma que el controlador responde correctamente con un código HTTP 404 cuando la cita solicitada no existe, garantizando un manejo consistente de los recursos inexistentes.

9.2.5. Prueba 5. Validación de identificadores inválidos en el controlador de citas

Esta prueba verifica que las operaciones de actualización y eliminación de citas validen correctamente el identificador recibido antes de ejecutar cualquier operación sobre la base de datos.

```
1  it('AppointmentController - Update y Delete fallan
2  devolviendo 400/404 al enviar IDs inválidos',
3  async () => {
4
5      const resUpdate =
6          await request(app)
7              .put('/api/appointments/123')
8              .send({
9                  estado: 'confirmada'
10             });
11
12     expect(resUpdate.status)
13         .toBe(400);
14
15     const resDelete =
16         await request(app)
17             .delete('/api/appointments/123');
18
19     expect(resDelete.status)
20         .toBe(404);
21
22 });
```

La correcta ejecución de esta prueba demuestra que el controlador valida apropiadamente los identificadores antes de realizar operaciones de modificación, evitando accesos inválidos y manteniendo la integridad de la aplicación.

9.2.6. Prueba 6. Validación de identificadores inválidos en el controlador de pacientes

Esta prueba verifica que las operaciones de actualización y eliminación de pacientes validen correctamente los identificadores recibidos antes de realizar modificaciones sobre la base de datos. Se comprueba que el sistema responda con los códigos HTTP apropiados cuando se utilizan identificadores inválidos.

```
1  it('PatientController - Update y Delete fallan
2  devolviendo 400/404 al enviar IDs inválidos',
3  async () => {
4
5      const resUpdate =
6          await request(app)
7              .put('/api/patients/123')
8              .send({
9                  telefono: '123'
10             });
11
12     expect(resUpdate.status)
13         .toBe(400);
14
15     const resDelete =
16         await request(app)
17             .delete('/api/patients/123');
18
19     expect(resDelete.status)
20         .toBe(404);
21
22 });
```

La correcta ejecución de esta prueba demuestra que el controlador valida adecuadamente los identificadores antes de procesar operaciones de actualización o eliminación, evitando modificaciones sobre recursos inexistentes o con formato inválido.

9.2.7. Prueba 7. Visualización de indicadores del panel principal

Esta prueba de componente verifica que el panel principal muestre correctamente los indicadores disponibles para un usuario con rol de recepcionista. Además, comprueba que únicamente se visualicen los elementos correspondientes a dicho perfil de usuario.


```
1  it('should render the dashboard widgets with correct counts',
2    () => {
3
4      mockRole = 'RECEPCIONISTA';
5
6      render(
7
8        <BrowserRouter>
9
10         <Dashboard />
11
12        </BrowserRouter>
13
14      );
15
16      expect(
17        screen.getByText(
18          'Pacientes registrados'
19        )
20      ).toBeInTheDocument();
21
22      expect(
23        screen.getByText(
24          'Citas totales'
25        )
26      ).toBeInTheDocument();
27
28      expect(
29
30        screen.queryByText(
31
32          /pendiente/i,
33
34          {
35            selector: '.section-title'
36          }
37
38        )
39
40      ).not.toBeInTheDocument();
41
42    });
```

La prueba confirma que el panel adapta correctamente su contenido según el rol del usuario autenticado, mostrando únicamente la información correspondiente a sus permisos.

9.2.8. Prueba 8. Manejo de errores al confirmar una cita

Esta prueba verifica que el panel administrativo gestione correctamente los errores producidos durante el proceso de confirmación de una cita. Para ello se simula un fallo en la actualización de la información y se comprueba que el usuario reciba la notificación correspondiente.

```
1  it('should handle error in confirm appointment',
2  async () => {
3
4      mockRole = 'ADMINISTRADOR';
5
6      mockUpdateAppointment
7          .mockRejectedValueOnce(
8              new Error('Update failed')
9          );
10
11     render(
12
13         <BrowserRouter>
14
15             <Dashboard />
16
17         </BrowserRouter>
18
19     );
20
21     const confirmBtn =
22         screen.getByRole(
23             'button',
24             { name: /Confirmar/i }
25         );
26
27     fireEvent.click(confirmBtn);
28
29     await waitFor(() => {
30
31         expect(window.alert)
32             .toHaveBeenCalledWith(
33                 'Update failed'
34             );
35
36     });
37
38 });
```

La correcta ejecución de esta prueba garantiza que la interfaz informa oportunamente al usuario cuando ocurre un error durante la confirmación de una cita, favoreciendo una adecuada gestión de excepciones.

9.2.9. Prueba 9. Visualización de estados vacíos en el panel

Esta prueba verifica que el panel principal muestre mensajes informativos cuando no existen citas registradas en el sistema. Para ello se simula un contexto sin pacientes, doctores ni citas disponibles.

```
1  it('should render empty states when no appointments exist',
2  () => {
3
4      mockRole = 'ADMINISTRADOR';
5
6      vi.mocked(useApp)
7          .mockReturnValueOnce({
8
9          patients: [],
10         appointments: [],
11         doctors: [],
12         loading: false,
13         updateAppointment:
14             mockUpdateAppointment
15
16         });
17
18     render(
19
20         <BrowserRouter>
21
22             <Dashboard />
23
24         </BrowserRouter>
25
26     );
27
28     expect(
29         screen.getByText(
30             'No hay citas pendientes'
31         )
32     ).toBeInTheDocument();
33
34     expect(
35         screen.getByText(
36             'Sin citas registradas'
37         )
38     ).toBeInTheDocument();
39
40 });
```

La prueba confirma que el sistema proporciona mensajes claros cuando no existen datos disponibles, mejorando la experiencia del usuario y evitando interfaces vacías o ambiguas.

9.2.10. Prueba 10. Visualización del indicador de carga

Esta prueba verifica que el panel principal muestre un indicador visual mientras la información necesaria para construir la interfaz se encuentra en proceso de carga.

```
1  it('should render loading spinner',
2  () => {
3
4      vi.mocked(useApp)
5          .mockReturnValueOnce({
6
7          patients: [],
8          appointments: [],
9          doctors: [],
10         loading: true,
11         updateAppointment:
12             mockUpdateAppointment
13
14         });
15
16     render(
17
18         <BrowserRouter>
19
20             <Dashboard />
21
22         </BrowserRouter>
23
24     );
25
26     expect(
27         document.querySelector(
28             '.spinner'
29         )
30     ).toBeInTheDocument();
31
32 });
```

La correcta ejecución de esta prueba demuestra que la interfaz proporciona retroalimentación visual durante la carga de información, informando al usuario que el sistema continúa procesando la solicitud y mejorando la percepción de respuesta de la aplicación.

10. REQ007 - Generación de Notificaciones

10.1. Descripción

El requisito de Generación de Notificaciones comprende las funcionalidades responsables de informar automáticamente a los usuarios sobre los cambios de estado de las citas médicas. Cada modificación realizada sobre una cita debe desencadenar el envío de la

notificación correspondiente, garantizando que pacientes y personal médico reciban información actualizada respecto a la confirmación, reprogramación, cancelación o finalización de la atención.

Para validar este requisito se ejecutaron pruebas de cobertura sobre la lógica del modelo de citas, verificando que cada transición de estado invoque el mecanismo de notificación correspondiente. Como resultado, se ejecutaron satisfactoriamente un total de cuatro casos de prueba.

Resultado general del requisito

- Total de pruebas ejecutadas: 4
- Pruebas aprobadas: 4
- Pruebas fallidas: 0
- Estado del requisito: Aprobado

10.2. Implementación

10.2.1. Prueba 1. Notificación al confirmar una cita

Esta prueba verifica que, al actualizar el estado de una cita a *confirmada*, el sistema ejecute automáticamente el mecanismo encargado de enviar la notificación correspondiente al paciente.

```
1  it('Update estado a confirmada dispara
2  sendAppointmentConfirmed',
3  async () => {
4
5      const a =
6          await AppointmentModel.create({
7
8              patientId: p._id,
9              doctorId: d._id,
10             fecha: '2030-10-11',
11             hora: '08:00'
12
13         });
14
15         await AppointmentModel.update(
16
17             a._id,
18
19             {
20                 estado: 'confirmada'
21             }
22
23         );
24
25     });
```

La correcta ejecución de esta prueba confirma que el sistema genera automáticamente la notificación de confirmación cuando una cita cambia a dicho estado, garantizando una comunicación oportuna con el usuario.

10.2.2. Prueba 2. Notificación al reprogramar una cita

Esta prueba verifica que el sistema envíe la notificación correspondiente cuando una cita es reprogramada. Además del cambio de estado, se actualizan la fecha y la hora de atención para comprobar que la información modificada sea procesada correctamente.

```
1  it('Update estado a reprogramada dispara
2  sendAppointmentRescheduled',
3  async () => {
4
5      const a =
6          await AppointmentModel.create({
7
8              patientId: p._id,
9              doctorId: d._id,
10             fecha: '2030-10-12',
11             hora: '09:00'
12
13         });
14
15         await AppointmentModel.update(
16
17             a._id,
18
19             {
20
21                 estado: 'reprogramada',
22                 fecha: '2030-10-13',
23                 hora: '10:00'
24
25             }
26
27         );
28
29     });
```

La prueba demuestra que el sistema detecta correctamente la reprogramación de una cita y activa el envío de la notificación correspondiente, informando al usuario sobre la nueva fecha y hora asignadas.

10.2.3. Prueba 3. Notificación al cancelar una cita

Esta prueba verifica que el sistema genere automáticamente la notificación correspondiente cuando una cita cambia al estado *cancelada*. El objetivo es comprobar que la lógica de actualización invoque el mecanismo encargado de informar al paciente sobre la cancelación de su cita.

```
1  it('Update estado a cancelada dispara
2  sendAppointmentCancelled',
3  async () => {
4
5      const a =
6          await AppointmentModel.create({
7
8              patientId: p._id,
9              doctorId: d._id,
10             fecha: '2030-10-11',
11             hora: '10:00'
12
13         });
14
15         await AppointmentModel.update(
16
17             a._id,
18
19             {
20                 estado: 'cancelada'
21             }
22
23         );
24
25     });
```

La correcta ejecución de esta prueba confirma que el sistema detecta el cambio de estado hacia *cancelada* y ejecuta el mecanismo de notificación correspondiente, asegurando que el usuario sea informado oportunamente sobre la cancelación de la cita.

10.2.4. Prueba 4. Notificación al completar una cita

Esta prueba verifica que, al finalizar una atención médica y actualizar el estado de la cita a *completada*, el sistema genere automáticamente la notificación correspondiente. De esta manera se valida el correcto funcionamiento del flujo asociado a la finalización del proceso de atención.

```
1  it('Update estado a completada dispara
2  sendAppointmentCompleted',
3  async () => {
4
5      const a =
6          await AppointmentModel.create({
7
8              patientId: p._id,
9              doctorId: d._id,
10             fecha: '2030-10-11',
11             hora: '12:00'
12
13         });
14
15         await AppointmentModel.update(
16
17             a._id,
18
19             {
20                 estado: 'completada'
21             }
22
23         );
24
25     });
```

La ejecución satisfactoria de esta prueba demuestra que el sistema genera automáticamente la notificación de finalización de la cita cuando esta cambia al estado *completada*, garantizando la correcta ejecución del proceso de comunicación asociado al cierre de la atención médica.